

WiFi Test Orchestrator

**Arquitectura multiplataforma, motor de tráfico, proceso de desarrollo
y extensibilidad**

Servidor Windows-first portable
Agentes Windows, Linux, Android e iOS

Documento técnico de arquitectura, implementación y mantenimiento

Versión 2.0 - 11 de julio de 2026

Preparado para Paolo Reynalde

Uso autorizado

Plataforma original para laboratorios y redes autorizadas. No constituye certificación de Wi-Fi Alliance ni réplica de software propietario.

Control del documento

Campo	Valor
Título	WiFi Test Orchestrator - Arquitectura, desarrollo y extensibilidad
Versión	2.0
Fecha	11 de julio de 2026
Autor/solicitante	Paolo Reynalde
Estado	Documento de diseño y guía de implementación
Servidor inicial	Windows 11 + Docker Desktop/WSL2; servicios Linux OCI
Agentes	Windows, Linux, Android e iOS
Paquete asociado	Prompts_Codex_WiFi_Test_Orchestrator_v2.0.zip

Historial de revisiones

Versión	Fecha	Cambios
1.0	11/07/2026	Arquitectura inicial servidor-agente y prompts base.
1.1	11/07/2026	Servidor Windows-first y agentes Windows/Linux/Android/iOS.
2.0	11/07/2026	Motor de tráfico desacoplado; iperf3, HTTP/native, latency-under-load, Flent, replay; SDK de plugins, versionado, migraciones, runbooks y 26 prompts.

Contenido

- 1. Resumen ejecutivo
- 2. Objetivos, alcance y exclusiones
- 3. Principios de diseño
- 4. Arquitectura general y topologías
- 5. Servidor Windows-first portable
- 6. Diseño de agentes por plataforma
- 7. Modelo de capacidades
- 8. Protocolo servidor-agente
- 9. Seguridad y modelo de amenazas
- 10. Modelo de datos y contratos
- 11. Orquestación de tareas
- 12. Arquitectura de plugins
- 13. Motor de generación de tráfico
- 14. Perfiles y servidores de tráfico
- 15. Latencia bajo carga y bufferbloat
- 16. Telemetría, eventos y sincronización temporal
- 17. Captura, análisis y replay
- 18. Adaptadores de infraestructura
- 19. Casos, suites, planes y campañas
- 20. Frontend y experiencia operativa
- 21. Artefactos y reportes
- 22. Observabilidad, calidad y CI/CD
- 23. Packaging, actualizaciones y portabilidad
- 24. Extensibilidad, versionado y migraciones
- 25. Paso a paso del desarrollo con Codex
- 26. Roadmap, riesgos y runbooks
- 27. Referencias oficiales
- Apéndice A. Catálogo de prompts
- Apéndice B. Prompts completos para Codex

1. Resumen ejecutivo

WiFi Test Orchestrator será una plataforma distribuida para administrar agentes, bancos de prueba, campañas, tráfico controlado, telemetría y evidencias. El servidor se ejecutará inicialmente en Windows 11 mediante Docker Desktop y WSL2, pero sus servicios permanecerán empaquetados como contenedores Linux OCI. Los endpoints se cubrirán mediante agentes nativos para Windows, Linux, Android e iOS, cada uno con capacidades diferentes y explícitas.

La decisión central de esta versión es separar el motor de tráfico del proveedor iperf3. iperf3 seguirá siendo la herramienta principal para throughput TCP/UDP en desktop, pero coexistirá con proveedores HTTP, sockets nativos, pruebas compuestas de latencia bajo carga, integración opcional con Flent y replay de PCAP restringido a nodos de laboratorio Linux. El dominio utilizará contratos comunes y registrará siempre proveedor, versión, método, restricciones y disponibilidad.

El producto será capability-driven. Un agente no recibe una prueba por pertenecer a una plataforma, sino porque declara una capability compatible, los permisos están disponibles, el contexto de foreground/background es válido y los límites de política se cumplen. Una función inexistente se representa como SKIPPED o BLOCKED con causa, no como una falla del equipo bajo prueba.

Decisión principal

iperf3 es el proveedor inicial de throughput, no la arquitectura. La arquitectura es un motor extensible de tráfico y pruebas con perfiles versionados, límites de seguridad y resultados normalizados.

Área	Decisión v2.0
Servidor	Windows-first para desarrollo; contenedores portables a Linux/cloud.
Desktop agents	Python compartido con adaptadores Windows/Linux y servicios nativos.
Android	Kotlin; foreground controlado, WorkManager para sincronización y permisos runtime.
iOS	Swift foreground-first; Network.framework/URLSession; background no garantizado.
Tráfico	TrafficGenerator abstracto; iperf3, HTTP, sockets, composite load y providers futuros.
Bufferbloat	Latency-under-load propio; Flent opcional en nodos compatibles.
Replay	Tcpreplay solo en Lab/Capture Nodes Linux aislados.
Extensión	Plugin manifests, semver, conformance tests, feature flags y migraciones.

2. Objetivos, alcance y exclusiones

2.1 Objetivos funcionales

- Inventariar dispositivos, interfaces, versiones y capabilities.
- Enrolar, autenticar, revocar y actualizar agentes.
- Crear casos, suites, planes y campañas reproducibles.
- Ejecutar pruebas de conectividad, DNS, HTTP, latencia, throughput, estabilidad y latencia bajo carga.
- Generar tráfico controlado hacia servidores registrados.
- Correlacionar resultados del endpoint con infraestructura y capturas.
- Conservar métricas, resultados crudos, logs, PCAP y reportes.
- Comparar modelos, firmware, drivers, configuraciones y plataformas con reglas de equivalencia.

2.2 Requisitos no funcionales

- Seguridad por diseño y mínimo privilegio.
- Tolerancia a desconexiones y ejecución at-least-once con efecto idempotente.
- Contratos versionados y compatibilidad hacia atrás.
- Observabilidad completa y troubleshooting documentado.
- Portabilidad del servidor y aislamiento de dependencias por plataforma.
- Capacidad de agregar plugins sin modificar el núcleo.
- Trazabilidad de cada medición y decisión PASS/FAIL.

2.3 Exclusiones iniciales

- Certificación oficial de Wi-Fi Alliance.
- Scan Wi-Fi continuo o control universal en móviles.
- Daemon permanente en iOS.
- Captura 802.11 desde teléfonos.
- Shell remoto arbitrario.
- Descifrado de tráfico sin autorización y material explícito.
- Uso de servidores públicos de throughput por defecto.
- Replay de PCAP desde agentes comunes.
- Equivalencia automática entre métodos diferentes de latencia o throughput.

3. Principios de diseño

Principio	Aplicación
Outbound-only	Los agentes abren HTTPS hacia el servidor; no requieren puertos entrantes.
Capability-driven	La asignación depende de capabilities, permisos, límites y contexto.
Datos honestos	null/unknown incluye source, availability y motivo; nunca se reemplaza por cero.
Native mobile	Android y iOS usan runtimes nativos; comparten contratos, no binarios desktop.
Immutable execution	Cada ejecución conserva snapshots de definición, thresholds, versiones y manifests.
Provider abstraction	El dominio no conoce comandos de iperf3, curl, netsh, iw o tcpreplay.
Safety by policy	Destinos, puertos, bitrate, duración y concurrencia están acotados.
Extensibilidad explícita	SDK, manifests, conformance tests y políticas de deprecación.
Portable server	Contenedores Linux, configuración por entorno y storage abstraído.
Evidence first	Cada resultado puede rastrearse hasta parámetros, raw output y artefactos.

4. Arquitectura general y topologías

4.1 Componentes

```

flowchart TB
    UI[Frontend React] --> API[FastAPI Control Plane]
    API --> DB[(PostgreSQL)]
    API --> REDIS[(Redis)]
    REDIS --> WORKERS[Workers]
    API --> ART[Artifact Store]
    WIN[Windows Agent] -->|HTTPS outbound| API
    LNX[Linux Agent] -->|HTTPS outbound| API
    AND[Android App] -->|HTTPS + push hint| API
    IOS[iOS App] -->|HTTPS + APNs hint| API
    TRAF[Traffic Server Pool] <-->|Data plane| WIN
    TRAF <-->|Data plane| AND
    CAP[Capture/Lab Node Linux] --> API
    INF[AP/GW/Controller Adapters] --> API
  
```

Plano	Responsabilidad	Ejemplos
Control	Usuarios, agentes, campañas, políticas, scheduling y auditoría.	FastAPI, PostgreSQL, Redis.
Data	Flujos de prueba entre agente y servidor de tráfico.	iperf3, HTTP, TCP/UDP nativo.
Telemetry	Muestras, eventos, availability, clocks y	RSSI, rates, path, CPU, battery.

Plano	Responsabilidad	Ejemplos
	series.	
Artifact	Logs, raw JSON, PCAP, CSV y PDF.	Filesystem/S3/MinIO.
Integration	Estado/configuración de infraestructura.	SNMP, REST, TR-181/USP.

4.2 Topologías previstas

Single-host demo

Servidor, PostgreSQL, Redis, workers, frontend y traffic server en Docker Desktop. Adecuado para desarrollo, no para pruebas de máxima capacidad.

Laboratorio distribuido

Servidor central y uno o más Traffic Nodes cercanos a los APs. Evita que el host de control sea cuello de botella.

Laboratorio con Capture Node

Agrega un Linux con NIC dedicada en modo monitor para correlacionar tramas 802.11 y eventos de roaming.

Multi-site

Servidor central y nodos de tráfico por sitio; políticas de affinity evitan medir WAN cuando la intención es caracterizar WLAN.

5. Servidor Windows-first portable

El host inicial recomendado es Windows 11 con WSL2 y Docker Desktop. El repositorio debe residir dentro del filesystem de WSL para evitar penalizaciones de I/O. Docker Compose levanta servicios Linux; scripts PowerShell ofrecen conveniencia al operador, pero no contienen lógica indispensable.

- No montar rutas absolutas de Windows en configuración de aplicación.
- Usar variables de entorno y secrets externos.
- Separar perfiles `dev`, `demo`, `test` y `production-like`.
- No publicar PostgreSQL/Redis en interfaces externas por defecto.
- Agregar health/readiness, backups y restore tests desde la primera release.
- Mantener manifests compatibles con despliegue futuro en Linux o Kubernetes.

```
Windows 11 host
Docker Desktop + WSL2
  nginx/caddy
  frontend
  backend API
  workers
  PostgreSQL
  Redis
  artifact store
  optional local traffic server
Native Windows Agent (fuera de Docker)
```

6. Diseño de agentes por plataforma

Capacidad	Windows	Linux	Android	iOS
Servicio persistente	Sí	Sí	Condicional/visible	No garantizado
SSID/BSSID	Generalmente disponible	Disponible	Permisos/versión	Condicional
RSSI	Según API/driver	Según driver	Disponible con límites	No universal
Scan	Sí	Sí	Throttling/permisos	No genérico
iperf3 externo	Sí	Sí	No en MVP	No en MVP
HTTP/native traffic	Sí	Sí	Sí	Sí
Captura IP	Npcap opcional	dumpcap/tcpdump	VpnService opcional	No general en MVP
Monitor 802.11	No objetivo	Capture Node	No	No
Ejecución larga	Servicio	Servicio	Foreground + límites	Foreground-first

6.1 Núcleo desktop

Windows y Linux comparten transporte, cola SQLite, task runner, plugin registry, contratos y manejo de artefactos. Las APIs del sistema se encapsulan en PlatformAdapter, WifiCollector, ProcessRunner, SecretStore y ServiceManager.

6.2 Android

La app debe declarar permisos mínimos, mostrar el estado de ejecución y respetar foreground service restrictions. WorkManager se usa para sincronización diferible, no como scheduler exacto. El servidor debe considerar el estado WAITING_FOR_FOREGROUND y expiraciones.

6.3 iOS

La app es foreground-first. Network.framework y URLSession realizan pruebas; BGTaskScheduler y APNs sirven para sincronización o hints, no garantizan ejecución inmediata. Las capabilities deben reflejar entitlements y condiciones reales.

7. Modelo de capacidades

El manifiesto de capacidades es el contrato operativo entre agente y orquestador. Debe versionarse y regenerarse ante cambios de permisos, app, OS, adaptadores o dependencias.

```
{
  "schema_version": "2.0",
  "agent_id": "00000000-0000-0000-0000-000000000001",
  "platform": "android",
  "platform_version": "15",
  "agent_version": "0.1.0",
  "generated_at": "2026-07-11T18:00:00Z",
  "capabilities": [
    {
      "id": "wifi.current_connection",
      "version": "1.0",
      "support": "conditional",
      "conditions": [
        "runtime_permission"
      ],
      "provider": "android.wifi",
      "limits": {}
    },
    {
      "id": "traffic.http.download",
      "version": "1.0",
      "support": "supported",
      "provider": "okhttp",
      "limits": {
        "max_duration_seconds": 1800,
        "max_parallel_streams": 8
      }
    },
    {
      "id": "traffic.iperf3.tcp",
      "version": "1.0",
      "support": "unsupported",
      "availability_reason": "binary_provider_not_installed"
    },
    {
      "id": "execution.background_continuous",
      "version": "1.0",
      "support": "conditional",
      "conditions": [
        "foreground_service_visible",
        "os_time_limits"
      ]
    }
  ]
}
```

Support	Significado
supported	Disponible sin condición adicional en el contexto actual.

Support	Significado
conditional	Disponible si se cumplen condiciones declaradas.
permission_required	Requiere permiso o privilegio aún no concedido.
user_interaction_required	Necesita acción visible del usuario.
unsupported	No existe para esa plataforma/versión.

El preflight de campaña combina capability, permisos, foreground, batería/thermal, red actual, límites de servidor y políticas de seguridad. El resultado se muestra antes de ejecutar.

8. Protocolo servidor-agente

8.1 Enrolamiento

1. Administrador genera token de enrolamiento de uso limitado.
2. Agente presenta token y fingerprint/metadata mínima.
3. Servidor crea agent_id y credencial individual.
4. Agente guarda secreto en DPAPI/keyring/Keystore/Keychain.
5. Agente publica capabilities y presencia.
6. Servidor permite rotación y revocación.

8.2 Task envelope

```
{
  "schema_version": "2.0",
  "task_id": "00000000-0000-0000-0000-000000000010",
  "execution_id": "00000000-0000-0000-0000-000000000011",
  "type": "traffic.run_profile",
  "type_version": "1.0",
  "created_at": "2026-07-11T18:00:00Z",
  "not_before": "2026-07-11T18:00:00Z",
  "expires_at": "2026-07-11T18:20:00Z",
  "idempotency_key": "00000000-0000-0000-0000-000000000012",
  "required_capabilities": [
    "traffic.tcp.throughput"
  ],
  "foreground_requirement": "required_on_mobile",
  "user_interaction_requirement": "possible",
  "parameters": {
    "traffic_profile_id": "tcp-downlink-4streams-30s",
    "traffic_profile_version": 1,
    "server_reservation_id": "00000000-0000-0000-0000-000000000013"
  }
}
```

8.3 Idempotencia y entrega

La entrega es at-least-once, pero el efecto debe ser idempotente. Cada task incluye idempotency key; el agente persiste claim, progreso, resultado y uploads. Un fencing token evita que un lease vencido publique como dueño actual.

Estado	Uso
QUEUED	Esperando elegibilidad.
OFFERED	Visible para un agente.
CLAIMED	Lease concedido.
RUNNING	Plugin activo.
WAITING_FOR_FOREGROUND	Mobile requiere app visible.
WAITING_FOR_USER	Falta permiso/acción.
COMPLETED/FAILED	Final técnico.
SKIPPED/BLOCKED	No aplicable o impedido por política/capability.

Estado	Uso
CANCELED/EXPIRED	Cancelado o fuera de ventana.

9. Seguridad y modelo de amenazas

Riesgo	Control principal
Abuso de tráfico	Allowlist de destinos, reservas, cuotas, bitrate/duración/concurrencia y auditoría.
Comando arbitrario	Plugins registrados; ProcessRunner allowlisted; sin shell libre.
Robo de credencial	Secret stores, rotación, revocación, TLS y scopes.
Replay de órdenes	Expiración, nonce/idempotency, firma o autenticidad del canal.
Captura excesiva	Filtros, max size/time, permisos y política de retención.
Plugin malicioso	Firma/allowlist, API versionada, conformance tests y rollout controlado.
Datos falsos	Source/availability, raw evidence, timestamps y validation.
Configuración de AP incorrecta	Snapshot, validate, permisos separados y rollback.
Supply chain	SBOM, firmas, hash, SAST/SCA y release gates.

Los límites deben evaluarse tanto en el servidor como en el agente. El servidor previene asignaciones abusivas; el agente actúa como última barrera ante tareas manipuladas o configuración stale.

10. Modelo de datos y contratos

Entidad	Responsabilidad
Agent / Device	Identidad, plataforma, versión, estado y asociación física/lógica.
CapabilitySnapshot	Manifiesto inmutable usado en preflight/ejecución.
TestPlugin / Provider	Tipo, versión, schemas, plataformas y permisos.
TestCase / Suite / Plan	Definiciones versionadas y publicables.
Campaign / Target	Ejecución planificada sobre conjunto de dispositivos.
Task / Lease / Event	Orquestación y máquina de estados.
Execution / Result	Snapshot, outcome, summary y evaluación.
MetricSeries / Sample	Series temporales normalizadas.
Artifact	Archivo, hash, media type, retención y acceso.
TrafficServer / Reservation	Capacidades y recursos del data plane.
ExternalDevice / Adapter	Infraestructura y fuente de telemetría.
AuditLog	Actor, acción, recurso, resultado y correlation.

Los contratos deben incluir `schema\_version`. Los cambios aditivos son preferidos. Cambios incompatibles requieren nueva versión, migración, compatibility window y test vectors para los cuatro runtimes.

10.1 Endpoints principales

Método	Ruta	Propósito
POST	/api/v1/auth/login	Autenticar usuario.
POST	/api/v1/agents/enrollment-tokens	Crear token.

Método	Ruta	Propósito
POST	/api/v1/agents/enroll	Enrolar agente.
POST	/api/v1/agents/{id}/presence	Heartbeat/presencia.
GET	/api/v1/agents/{id}/tasks	Obtener ofertas.
POST	/api/v1/tasks/{id}/claim	Reclamar lease.
POST	/api/v1/executions/{id}/events	Progreso/eventos.
POST	/api/v1/executions/{id}/result	Resultado idempotente.
POST	/api/v1/artifacts/initiate	Upload multipart.
POST	/api/v1/campaigns	Crear campaña.
GET	/api/v1/traffic-servers	Pool disponible.

11. Orquestación de tareas

El orquestador resuelve elegibilidad, reserva recursos, crea task envelopes y controla leases. Debe ser resistente a reinicios de API, workers, Redis y agentes.

- Preflight de capabilities y política.
- Reserva de Traffic Server antes de ofrecer una prueba de carga.
- Lease con timeout, renewal y fencing token.
- Progreso ordenado mediante sequence number.
- Cancelación cooperativa y cleanup obligatorio.
- Expiración clara para mobile.
- Control de concurrencia por campaña, agente, laboratorio y servidor.
- Eventos inmutables para reconstruir la historia.

```
Campaign Target -> Preflight -> Reserve resources -> QUEUED
-> OFFERED -> CLAIMED -> RUNNING
-> COMPLETED | FAILED | CANCELED | EXPIRED
-> SKIPPED/BLOCKED when capability/policy prevents execution
```

12. Arquitectura de plugins

El núcleo debe depender de interfaces, no de herramientas. Cada plugin publica un manifest con versión semántica, API version, schemas, capabilities, plataformas, permisos y límites.

```
{
  "schema_version": "2.0",
  "plugin_id": "com.example.traffic.iperf3",
  "plugin_version": "1.0.0",
  "plugin_api_version": "2.0",
  "kind": "traffic_generator",
  "platforms": [
    "windows",
    "linux"
  ],
  "capabilities": [
    "traffic.iperf3.tcp",
    "traffic.iperf3.udp"
  ],
  "config_schema": "schemas/iperf3_config.schema.json",
  "result_schema": "schemas/iperf3_result.schema.json",
  "permissions": [
    "process.spawn.allowlisted"
  ],
  "resource_limits": {
    "max_duration_seconds": 3600,
    "max_parallel_streams": 32
  },
  "signature": {
    "algorithm": "ed25519",
    "key_id": "dev-example",
    "value": "placeholder-for-test-fixture"
  }
}
```

Interfaz	Responsabilidad
TestPlugin	Preparar, ejecutar, evaluar y limpiar una prueba.
TrafficGenerator	Generar carga y emitir intervalos normalizados.
LatencyProbe	Medir RTT/tiempo con método declarado.
CaptureProvider	Capturar tráfico bajo políticas.
InfrastructureAdapter	Leer/configurar infraestructura autorizada.
ReportRenderer	Renderizar formatos y templates.

El lifecycle recomendado es validate -> preflight -> prepare -> start -> progress -> stop/cancel -> collect -> evaluate -> cleanup. `cleanup` debe ser idempotente y ejecutarse aun cuando falle la fase principal.

13. Motor de generación de tráfico

13.1 Contrato TrafficGenerator

```
interface TrafficGenerator {
    validate(profile, agentCapabilities, serverCapabilities)
    prepare(context) -> PreparedRun
    start(preparedRun) -> RunHandle
    streamProgress(runHandle) -> IntervalMetric[]
    stop(runHandle, reason)
    collect(runHandle) -> RawAndNormalizedResult
    cleanup(runHandle)
}
```

13.2 Proveedores

Proveedor	Plataformas	Uso	Limitación
iperf3	Windows/Linux	TCP/UDP, reverse, bidir, streams.	Binario externo; no es tráfico de aplicación.
HTTP	Todas	Download/upload y timings de app.	Depende de servidor HTTP y cache control.
Native sockets	Todas; clave en mobile	TCP/UDP controlado y portable.	Protocolo propio; no equivale a iperf3.
Composite load	Todas	Carga + probe simultáneo.	Requiere sincronización y recursos.
Flent adapter	Linux/desktop opcional	Bufferbloat y pruebas agregadas.	Dependencias externas; no MVP obligatorio.
Tcp replay	Lab Node Linux	Replay de PCAP.	Alto privilegio; red aislada; no stateful app traffic.

13.3 iperf3

iperf3 es el proveedor principal de throughput desktop porque soporta TCP/UDP, múltiples streams, reverse, bidirectional y salida JSON. Debe ejecutarse mediante ProcessRunner allowlisted, detectar versión, conservar raw output y aplicar kill-tree/cleanup. El servidor de prueba impone reservas y límites.

13.4 Mobile

Android e iOS usarán HTTP y sockets nativos en el MVP. Una integración futura de libiperf exige revisión de licencia, estabilidad, build, App Store/MDM y compatibilidad; no debe descargarse un binario remoto ni ejecutarse código no verificado.

13.5 Métricas normalizadas

- throughput_bps por intervalo y resumen.
- bytes enviados/recibidos.
- duration efectiva, warmup y cooldown.
- jitter/loss para UDP cuando exista.
- retransmissions para TCP cuando el provider lo reporte.

- provider id/version/method.
- CPU, thermal, battery y path como contexto, no como throughput.
- timestamps monotónicos y wall-clock quality.

14. Perfiles y servidores de tráfico

14.1 TrafficProfile

```
{
  "schema_version": "2.0",
  "profile_id": "tcp-downlink-4streams-30s",
  "profile_version": 1,
  "name": "TCP downlink - 4 streams - 30 s",
  "generator": {
    "capability": "traffic.tcp.throughput",
    "preferred_provider": "iperf3",
    "fallback_policy": "explicit_only"
  },
  "direction": "downlink",
  "protocol": "tcp",
  "duration_seconds": 30,
  "parallel_streams": 4,
  "warmup_seconds": 3,
  "cooldown_seconds": 3,
  "network": {
    "ip_version": "any",
    "dscp": null
  },
  "limits": {
    "max_bitrate_bps": null
  },
  "thresholds": {
    "min_average_bps": 500000000
  }
}
```

Los perfiles deben ser publicados e inmutables por versión. Un cambio de streams, duración, bitrate o threshold crea una nueva versión. El fallback de provider es explícito: el operador debe conocer si una prueba se ejecutó con iperf3, HTTP o sockets.

14.2 Perfiles recomendados

Perfil	Configuración inicial	Objetivo
TCP single	30 s, 1 stream, UL/DL	Eficiencia de un flujo.
TCP parallel	30 s, 4 streams, UL/DL	Capacidad agregada.
TCP bidir	30 s simultáneo	Competencia UL/DL.
UDP staircase	25/50/75/90 % estimado	Loss/jitter frente a carga.
HTTP download	Archivo controlado + checksum	Experiencia de aplicación.
HTTP upload	Payload controlado	Subida de aplicación.
Latency under load	Baseline + load + recovery	Bufferbloat/colas.
Stability	Carga periódica 15-60 min	Desconexiones y degradación.

14.3 Traffic Server Pool

- Servidor registrado con capabilities y health.
- Reserva con puerto/token, ventana y capacidad.
- Affinity por sitio/laboratorio.
- Máximo de sesiones, bitrate y duración.
- No usar Internet público por defecto.
- Métricas de CPU/NIC/drops para detectar cuello del servidor.
- Drenaje para mantenimiento.
- Certificados y autenticación donde aplique.

15. Latencia bajo carga y bufferbloat

La prueba compuesta ejecuta un probe independiente antes, durante y después de la carga. No se limita a un único promedio: conserva series, fases y distribución.

```
baseline (10 s) -> warmup -> load (30 s) -> cooldown -> recovery (10 s)
||
ICMP/TCP/HTTP latency probe every N ms
```

- Mediana, p95 y p99 de latencia por fase.
- Aumento absoluto y relativo respecto de baseline.
- Jitter y pérdida.
- Tiempo de recuperación.
- Identificación del método de probe.
- Series de throughput y latency alineadas.
- Severidad configurable; no universal.

Flent puede incorporarse como adapter opcional en Linux para pruebas agregadas como RRUL. La plataforma debe importar su JSON y metadatos, pero mantener un runner propio para que el MVP funcione sin Flent.

Interpretación

Una latencia elevada bajo carga evidencia formación de colas, pero no identifica por sí sola si la causa está en Wi-Fi, gateway, WAN o servidor. El reporte debe correlacionar contexto y evitar diagnósticos automáticos no sustentados.

16. Telemetría, eventos y sincronización temporal

Cada muestra conserva `collected\_at`, `received\_at`, sequence, source, availability y metadatos de reloj. Para intervalos de una ejecución se prioriza reloj monotónico local; el wall clock se usa para correlación y se acompaña de clock skew/quality.

Evento	Criterio
Association	La interfaz pasa a asociada.
Disconnect	Se pierde asociación/path.
BSSID change	Cambio observable del BSSID actual.
Channel change	Cambio de canal/frecuencia informado.
IP/path change	Cambio de dirección o transporte.
Telemetry gap	Intervalo mayor al umbral sin muestras.
Agent restart	Nueva sesión/boot id.
Clock jump	Diferencia anormal entre monotonic y wall clock.

- Batches idempotentes y comprimidos.
- Deduplicación y debounce de eventos.
- Retención configurable y downsampling.
- Particionamiento por tiempo/dispositivo.
- Export CSV/JSON.
- Métricas de ingest lag, rejects y backlog.

17. Captura, análisis y replay

Nivel	Plataformas	Alcance
Captura IP endpoint	Windows/Linux	Tráfico del endpoint; Npcap/dumpcap/tcpdump.
Captura IP app	Android opcional	VpnService con consentimiento y alcance de la app/dispositivo.
Captura iOS	Limitada	Sin captura general en MVP; herramientas de desarrollo separadas.
Monitor 802.11	Capture Node Linux	Radiotap, management/control/data visibles en canal.

Nivel	Plataformas	Alcance
Replay PCAP	Lab Node Linux	Tcp replay bajo aislamiento, policy y aprobación.

Captura y replay son plugins de alto privilegio. Requieren perfiles, permisos, límites, auditoría y cleanup. Un PCAP se valida por hash, tamaño y tipo. El replay nunca debe usar una ruta pública o un destino no autorizado.

El análisis básico puede extraer DNS, DHCP, TCP retransmissions, handshakes y errores. Las tramas 802.11 solo se analizan cuando la captura realmente contiene radiotap/802.11; no se infieren desde una captura Ethernet/IP.

18. Adaptadores de infraestructura

Los adaptadores permiten correlacionar la experiencia del endpoint con estado de AP, gateway, ONT o controlador. La primera implementación debe ser de solo lectura; las escrituras se agregan con permisos y rollback.

- SNMP, REST, TR-181/USP y APIs de controlador detrás de una interfaz común.
- Freshness y source por dato.
- Secrets en secret store.
- Timeout, retry y circuit breaker.
- Correlación por MAC/agent/device con confidence.
- Snapshot before/after y rollback para cambios.
- No usar SSH como consola remota genérica.

19. Casos, suites, planes y campañas

Objeto	Descripción
TestCase	Plugin + parámetros + thresholds + requisitos.
Suite	Conjunto ordenado de casos.
Plan	Suites versionadas para modelo/proyecto.
Campaign	Ejecución de un plan sobre targets y ventana.
Execution	Instancia inmutable con snapshots.
Baseline	Conjunto de referencia compatible.

Las campañas heterogéneas deben mostrar preflight por target. Un Android puede ejecutar HTTP/native y un Windows iperf3; el informe puede agruparlos, pero no debe comparar throughput como idéntico salvo una regla explícita.

- Serial/paralelo y max concurrency.
- Repeticiones y warmup.
- Pausa, reanudación y cancelación.
- Rerun de fallas seleccionadas.
- Thresholds inmutables por ejecución.
- Comparison engine con método, provider, sample size y dispersion.

20. Frontend y experiencia operativa

- Inventario con plataforma, estado, versión y capabilities.
- Detalle de telemetría con source/availability.
- Builder de campañas con preflight.
- Indicaciones de foreground/user interaction en mobile.
- Progreso en tiempo real y timeline de eventos.
- Gráficos de throughput, latencia, RSSI, rates y loss.
- Comparador con filtros por método/provider/firmware.
- Artefactos, auditoría y administración RBAC.
- Timezone configurable y exports.

Los dashboards deben mostrar SKIPPED y BLOCKED como estados de primera clase. Ocultarlos produce métricas engañosas de éxito y dificulta distinguir incompatibilidad de falla real.

21. Artefactos y reportes

Cada artefacto posee manifest con tipo, tamaño, SHA-256, media type, execution, source y retention class. Los uploads deben reanudarse, verificar hash y aplicar cuotas.

Reporte	Contenido mínimo
Ejecutivo	Objetivo, cobertura, resultado global y riesgos.
Técnico	Dispositivos, versions, parámetros, thresholds, series y fallas.
Metodología	Provider, método, disponibilidad, foreground y limitaciones.
Evidencias	Links autorizados a raw JSON, logs y capturas.
Comparación	Baseline, muestra, método y diferencias.

Formatos: HTML, PDF, CSV y JSON. El PDF no debe contener secretos ni PCAP completo. El acceso a artefactos se audita y puede usar URLs firmadas de corta duración.

22. Observabilidad, calidad y CI/CD

- Logs estructurados con correlation IDs y redacción.
- Prometheus: API latency, task queues, leases, ingest, traffic reservations, artifacts y agent states.
- OpenTelemetry para trazas de flujos críticos.
- Dashboards y alertas con SLOs.
- Pytest/Vitest/Playwright/XCTest/Android tests.
- Contract tests cross-language.
- SAST, SCA, secret scanning, container scanning y SBOM.
- Load tests de ingestión, agent storm y uploads.
- Chaos tests de Redis/PostgreSQL/worker/agent.
- Backup/restore probado en CI o entorno programado.

23. Packaging, actualizaciones y portabilidad

Componente	Distribución inicial	Actualización
Servidor	Docker Compose versionado	Pull de imágenes firmadas + migraciones.
Windows Agent	MSI/MSIX o instalador + Windows Service	Manifest firmado, canary y rollback.
Linux Agent	DEB; RPM futuro	Repositorio/paquete firmado.
Android	APK/AAB interno, MDM o Play según política	Canal de plataforma.
iOS	Development, TestFlight o MDM	Canal Apple/MDM.
Plugins	Empaquetados con release o repositorio interno firmado	Feature flags y compatibility checks.

Las actualizaciones deben considerar una ventana de compatibilidad N/N-1 entre servidor y agentes. Mobile no utiliza el mismo mecanismo de auto-update que desktop. Las migraciones de base deben coordinarse con rolling upgrades y rollback.

24. Extensibilidad, versionado y migraciones

24.1 Cómo agregar una funcionalidad

1. Crear feature proposal con problema, alcance, riesgos y plataforma.
2. Agregar o actualizar ADR si cambia arquitectura.
3. Definir capability y schemas versionados.
4. Implementar plugin detrás de interfaz existente.
5. Agregar conformance tests y golden fixtures.
6. Documentar permisos, límites, seguridad y observabilidad.
7. Agregar migración y rollback si cambia persistencia.
8. Publicar detrás de feature flag/canary.

9. Actualizar changelog, compatibility matrix y runbook.

24.2 Compatibilidad

- Semver para plugins y APIs internas.
- schema_version para mensajes.
- Cambios aditivos por defecto.
- Nueva versión para breaking changes.
- Conservar adaptadores de lectura para formatos antiguos durante la ventana.
- No recalcular resultados históricos con lógica nueva sin crear una vista derivada explícita.
- Deprecation warning, fecha de sunset y plan de migración.

24.3 Migraciones

Cada migración debe definir precondiciones, upgrade, validación, downgrade o punto de no retorno, impacto de disponibilidad, backup requerido y compatibilidad de versiones. Las migraciones de datos voluminosas deben separarse de DDL y ejecutarse de forma resumible.

25. Paso a paso del desarrollo con Codex

Codex debe ejecutarse desde la raíz del repositorio y leer AGENTS.md antes de cada tarea. Los prompts del ZIP están diseñados como fases con gates y commits independientes.

Fase	Nombre	Salida principal
0	Preparación	Instalar Git, WSL2, Docker Desktop, VS Code/Codex, Python, Node. Android Studio para Android y un Mac/Xcode para iOS.
1	Arquitectura	Prompt 01: diagramas, ADRs, threat model y capability matrix.
2	Base ejecutable	Prompt 02: monorepo, Compose, backend/frontend y simulador.
3	Dominio/Auth	Prompt 03: DB, migraciones, RBAC y auditoría.
4	Contratos	Prompt 04: OpenAPI, schemas, enrolamiento y manifests.
5	Core desktop	Prompt 05: cola SQLite, task runner y adapters.
6-9	Agentes	Windows, Linux/Capture Node, Android e iOS. Paralelizable luego de contratos.
10	Orquestación	Leases, mobile delivery, states y concurrency.
11	Probes	Latency, DNS, HTTP, connectivity y stability.
12	Traffic engine	Contratos, profiles, provider registry y policy.
13	iperf3	Provider desktop y pool de servers.
14	HTTP/native	Providers cross-platform y mobile.
15	Load latency	Composite runner, bufferbloat y Flent opcional.
16	Telemetry	Ingestion, events, time sync y retention.
17	Campaigns	Versionado, snapshots y comparison.
18	Frontend	Workflows completos y dashboards.
19	Artifacts	Captura IP y reportes.
20	Replay	Lab Nodes y Tcpreplay seguro.
21	Adapters	Infraestructura externa.
22	Extension SDK	Plugins, versioning, migrations y flags.
23	Hardening	Security, observability y CI.
24	Packaging	Instaladores, firmas y updates.

Fase	Nombre	Salida principal
25	Docs	Runbooks, troubleshooting y soporte.
26	Release	E2E heterogéneo, restore, upgrade y RC.

25.1 Rutina por fase

```
git status
git add .
git commit -m "checkpoint: before phase XX"

# Ejecutar el prompt con Codex, revisar plan y diff.
# Ejecutar linters, type checks, tests, contract tests y migraciones.

git add .
git commit -m "feat: complete phase XX"
```

25.2 Handoff obligatorio

- Archivos modificados.
- Decisiones/ADRs.
- Comandos y resultados.
- Tests y cobertura relevante.
- Cambios de contrato/migración.
- Riesgos y limitaciones.
- Rollback.
- Próximos bloqueos.

26. Roadmap, riesgos y runbooks

26.1 Roadmap

Release	Contenido
MVP 0.1	Servidor, auth, agentes simulados, Windows/Linux básicos, ping/DNS/HTTP, iperf3 y campañas.
0.2	Android/iOS foreground, traffic providers nativos, telemetry y dashboards.
0.3	Latency-under-load, artifacts, capture IP y reportes completos.
0.4	Capture Node, Flent, adapters externos y comparison avanzado.
1.0	Packaging firmado, updates, E2E, runbooks, SLOs y support matrix.
Post-1.0	QUIC/WebRTC, Wi-Fi 7 specific tests, TRex, distributed scheduling y analytics.

26.2 Riesgos

Riesgo	Mitigación
Limitaciones mobile cambian	Capability/version matrix, feature flags y revisión por OS target.
Servidor de tráfico es cuello	Health/capacity metrics, nodos dedicados y affinity.
Métodos no comparables	Etiquetado y rules de equivalencia.
Clock drift	Monotonic time, skew estimation y quality.
Driver/API inconsistente	Source/availability, fallbacks encapsulados y fixtures localizados.
Abuso o saturación	Policy, quotas, reservations y audit.
Complejidad de plugins	SDK, conformance tests y deprecation policy.
Datos voluminosos	Retention, partitioning, downsampling y artifact tiers.

26.3 Runbooks mínimos

- Agente offline o no enrola.
- Tarea atascada/lease vencido.
- Traffic Server saturated.
- High ingest lag/backlog.
- Artifact upload/hash mismatch.
- Clock skew excesivo.
- Android no inicia foreground service.
- iOS no recibe/ejecuta background hint.
- iperf3 process/port huérfano.
- Capture Node pierde interfaz/canal.
- Backup/restore.
- Rollback de release.

27. Referencias oficiales

OpenAI Codex CLI: <https://developers.openai.com/codex/cli>

OpenAI - AGENTS.md: <https://developers.openai.com/codex/guides/agents-md>

Docker Desktop WSL2: <https://docs.docker.com/desktop/features/wsl/>

Android Wi-Fi permissions: <https://developer.android.com/develop/connectivity/wifi/wifi-permissions>

Android Wi-Fi scanning: <https://developer.android.com/develop/connectivity/wifi/wifi-scan>

Android foreground service types: <https://developer.android.com/develop/background-work/services/fgs/service-types>

Android foreground service restrictions: <https://developer.android.com/develop/background-work/services/fgs/restrictions-bg-start>

Android foreground service timeouts: <https://developer.android.com/develop/background-work/services/fgs/timeout>

Apple Network framework: <https://developer.apple.com/documentation/network>

Apple BGTaskScheduler: <https://developer.apple.com/documentation/backgroundtasks/bgtaskscheduler>

Apple Access Wi-Fi Information entitlement: <https://developer.apple.com/documentation/bundleresources/entitlements/com.apple.developer.networking.wifi-info>

iperf3 documentation: <https://software.es.net/iperf/>

iperf3 invocation options: <https://software.es.net/iperf/invoking.html>

Flent documentation: <https://flent.org/>

Tcpreplay documentation: <https://tcpreplay.appneta.com/>

Referencias consultadas el 11 de julio de 2026. Las APIs y restricciones móviles cambian por versión; la implementación debe validar documentación vigente al actualizar target SDK, Xcode o sistema operativo.

Apéndice A. Catálogo de prompts

Nº	Archivo	Fase
01	01_architecture_and_adrs.md	Arquitectura, ADRs y modelo de amenazas
02	02_windows_first_server_and_monorepo.md	Monorepo y servidor Windows-first portable
03	03_backend_domain_database_auth.md	Dominio backend, base de datos, autenticación y RBAC
04	04_contracts_capabilities_enrollment.md	Contratos, capabilities y enrolamiento de agentes
05	05_desktop_agent_core.md	Núcleo compartido de agentes desktop
06	06_windows_agent.md	Agente Windows

Nº	Archivo	Fase
07	07_linux_agent_and_capture_node.md	Agente Linux y Capture Node
08	08_android_agent.md	Agente Android nativo
09	09_ios_agent.md	Agente iOS/iPadOS nativo
10	10_task_orchestration_and_mobile_delivery.md	Orquestación, leases y entrega mobile
11	11_basic_network_probes.md	Pruebas básicas de red y conectividad
12	12_traffic_engine_architecture.md	Arquitectura del motor de tráfico y perfiles versionados
13	13_iperf3_provider_and_test_server_pool.md	Proveedor iperf3 y pool de servidores de prueba
14	14_http_and_native_socket_traffic.md	Tráfico HTTP y sockets nativos multiplataforma
15	15_latency_under_load_bufferbloat_fleit.md	Latencia bajo carga, bufferbloat y Flent
16	16_telemetry_wifi_events_and_time_sync.md	Telemetría, eventos Wi-Fi y sincronización temporal
17	17_campaigns_plans_and_result_comparison.md	Casos, suites, planes, campañas y comparación
18	18_frontend_dashboards_and_operator_workflows.md	Frontend, dashboards y flujos operativos
19	19_artifacts_packet_capture_and_reports.md	Artefactos, captura IP y reportes
20	20_pcap_replay_and_specialized_lab_nodes.md	Replay de PCAP y nodos especializados de laboratorio
21	21_external_device_adapters.md	Adaptadores de AP, gateways, ONTs y controladores
22	22_plugin_sdk_versioning_and_migrations.md	SDK de plugins, versionado y migraciones
23	23_security_observability_and_ci.md	Seguridad, observabilidad y CI/CD
24	24_packaging_distribution_and_updates.md	Packaging, distribución y actualizaciones
25	25_documentation_runbooks_and_supportability.md	Documentación, runbooks y soporte futuro
26	26_end_to_end_release_candidate.md	Validación end-to-end y release candidate

Uso recomendado

Copiar 00_AGENTS_master_context.md como AGENTS.md. Ejecutar los prompts en orden y no paralelizar agentes hasta estabilizar contratos en el Prompt 05.

Apéndice B. Prompts completos para Codex

Las páginas siguientes reproducen el contenido del paquete ZIP. Los archivos Markdown del ZIP son la fuente práctica para copiar y ejecutar en Codex.

AGENTS.md - Contexto maestro de WiFi Test Orchestrator

Actuá como arquitecto de software senior, desarrollador full stack, especialista en redes Wi-Fi IEEE 802.11, automatización de pruebas, aplicaciones móviles, sistemas distribuidos y seguridad.

1. Propósito

Construir una plataforma original para automatizar, ejecutar, observar y administrar pruebas Wi-Fi en laboratorios, pilotos y redes autorizadas. La solución puede inspirarse en capacidades generales de plataformas profesionales de testing, pero no debe copiar código, interfaces, nombres internos, documentación confidencial ni procesos propietarios de DEKRA, Wi-Fi Alliance u otros proveedores.

El producto debe permitir:

- Inventariar agentes, dispositivos de infraestructura y bancos de prueba.
- Ejecutar pruebas repetibles de conectividad, latencia, throughput, estabilidad y comportamiento de aplicaciones.
- Generar tráfico sintético controlado mediante proveedores intercambiables.

- Correlacionar resultados del endpoint con telemetría de AP, gateway, controlador o Capture Node.
- Gestionar casos, suites, planes, campañas, thresholds, artefactos y reportes.
- Conservar trazabilidad suficiente para comparar hardware, firmware, drivers, sistemas operativos y configuraciones.

2. Alcance de plataformas

Servidor

- Desarrollo y despliegue inicial en Windows 11 con Docker Desktop y WSL2.
- Servicios ejecutados como contenedores Linux OCI.
- Portabilidad obligatoria a Linux, macOS y entornos cloud sin cambiar el dominio ni los contratos.
- No depender de rutas, servicios o APIs exclusivas del host Windows.

Agentes

- Windows 10/11: agente desktop como Windows Service.
- Linux: agente desktop como servicio systemd y rol opcional de Capture Node.
- Android: aplicación nativa Kotlin, con Activity, Foreground Service y WorkManager según el lifecycle permitido.
- iOS/iPadOS: aplicación nativa Swift, foreground-first; BackgroundTasks, URLSession background y APNs solo como mecanismos permitidos y no como garantía de ejecución inmediata.

No asumir paridad de capacidades entre plataformas. El servidor debe usar un modelo capability-driven y explicar SKIPPED/BLOCKED cuando una función no exista o requiera interacción.

3. Arquitectura obligatoria

- Backend: Python 3.12+, FastAPI, SQLAlchemy 2, Alembic, Pydantic 2.
- PostgreSQL como fuente de verdad transaccional.
- Redis para coordinación, locks, rate limits y cola.
- Celery u otro worker compatible detrás de una interfaz de ejecución.
- Frontend React + TypeScript + Vite.
- Agentes Windows/Linux en Python con SQLite local y adaptadores por OS.
- Android en Kotlin, Jetpack, Room, WorkManager, Retrofit/OkHttp y APIs de red nativas.
- iOS en Swift/SwiftUI, URLSession, Network.framework, BackgroundTasks y Keychain.
- Contratos compartidos mediante OpenAPI, JSON Schema, ejemplos y contract tests; no compartir runtime a la fuerza.
- Artefactos detrás de una abstracción compatible con filesystem local y S3/MinIO.
- Comunicación outbound-only desde agentes hacia el servidor mediante HTTPS.

4. Separación de dominios

Mantener límites explícitos entre:

1. Control plane: usuarios, agentes, inventario, campañas, scheduling, políticas y auditoría.
2. Data plane de pruebas: tráfico generado entre endpoints y servidores de prueba autorizados.
3. Telemetry plane: muestras periódicas, eventos, availability y metadatos de medición.
4. Artifact plane: logs, PCAP/PCAPNG, JSON crudo, CSV, reportes y hashes.
5. Integration plane: adaptadores de AP, gateway, ONT, controlador, SNMP, TR-181/USP o APIs externas.

5. Motor de pruebas y tráfico

No acoplar el producto a iperf3. Definir interfaces versionadas:

- `TestPlugin`
- `TrafficGenerator`
- `LatencyProbe`
- `CaptureProvider`
- `InfrastructureAdapter`
- `ReportRenderer`

Proveedores iniciales:

- iperf3 para TCP/UDP en Windows/Linux.
- HTTP upload/download con timings en todas las plataformas.
- Sockets TCP/UDP nativos para Android/iOS y fallback desktop.
- Ping ICMP donde sea posible; TCP/HTTP latency donde no lo sea.
- Latency-under-load combinando generador de tráfico y probe independiente.
- Flent opcional en Linux/desktop para pruebas agregadas de bufferbloat.
- Tcpreplay únicamente en nodos Linux especializados, con allowlist, aislamiento y redes autorizadas.

Todos los resultados deben conservar proveedor, versión, método, dirección, protocolo, streams, bitrate solicitado, límites, servidor, timestamps, capacidad disponible y contexto de plataforma.

6. Reglas de seguridad

- Uso exclusivo en redes y dispositivos autorizados.
- Prohibido implementar shell remoto arbitrario.
- Cada plugin y comando debe estar registrado en una allowlist firmada/versionada.
- Las tareas deben incluir UUID, tipo, versión de esquema, parámetros validados, expiración, idempotency key y capacidades requeridas.
- TLS obligatorio fuera de desarrollo; autenticación individual por agente; revocación y rotación.
- Secretos cifrados y nunca visibles en logs, excepciones, reportes o telemetría.
- RBAC mínimo: Administrator, Test Manager, Operator, Viewer y Agent.
- Auditoría inmutable para acciones sensibles.
- Rate limits, cuotas, límites de duración, ancho de banda, concurrencia y destino para generación de tráfico.
- Capturas limitadas por tiempo, tamaño, interfaz y filtro; sin descifrado o evasión de sandbox/permisos.
- Tcpreplay no debe estar disponible para endpoints comunes ni para destinos arbitrarios.
- Toda modificación de infraestructura requiere snapshot previo, permisos específicos, validación y rollback.

7. Fidelidad técnica Wi-Fi

- No inventar métricas. `null` no significa cero.
- Cada campo medido debe incluir `source`, `availability`, `confidence` y, cuando aplique, `availability\_reason`.
- Diferenciar telemetría del SO, contadores de interfaz, captura IP, captura 802.11 y telemetría de infraestructura.
- Un endpoint no observa todas las tramas de terceros.
- Roaming medido por endpoint es observado/inferido salvo captura sincronizada.
- iOS no debe prometer scan genérico de redes, monitor mode, daemon permanente, RSSI universal ni control arbitrario de perfiles.
- Android debe respetar permisos runtime, scan throttling, foreground service restrictions y APIs soportadas de Suggestion/Specifier.
- Las comparaciones deben indicar si los métodos son equivalentes, aproximados o no comparables.

8. Modelo de capacidades

Cada agente publica un manifiesto versionado con:

- capability id y versión.
- soporte: supported, unsupported, conditional, permission_required o user_interaction_required.
- restricciones de foreground/background.
- límites de duración, throughput, tamaño y concurrencia.
- proveedor y versión.
- permisos/entitlements necesarios.
- razones de indisponibilidad.

Ejemplos: `wifi.current\_connection`, `wifi.scan`, `wifi.rssi`, `network.icmp\_ping`, `network.tcp\_probe`, `traffic.iperf3.tcp`, `traffic.http.download`, `traffic.latency\_under\_load`, `capture.ip`, `capture.monitor\_80211`, `replay.pcap`, `execution.background\_continuous`.

9. Contratos y compatibilidad

- Versionar OpenAPI y JSON Schemas.
- Usar `schema\_version` en mensajes persistidos o enviados.
- Cambios aditivos por defecto.
- Cambios incompatibles requieren nueva versión, migración y ventana de compatibilidad.
- Mantener golden test vectors y contract tests para Python, TypeScript, Kotlin y Swift.
- No cambiar semántica histórica al modificar thresholds o plugins.
- Cada ejecución conserva snapshot inmutable de parámetros, thresholds, manifests y versiones.

10. Calidad y documentación

- Código y nombres técnicos en inglés; documentación funcional principal en español.
- Type hints, validación estricta, errores estructurados y correlation IDs.
- Tests unitarios, integración, contrato, E2E, resiliencia y seguridad.
- Linters: Ruff/Black/MyPy, ESLint/Prettier, ktlint/detekt, SwiftLint.
- Cada feature actualiza README, referencia de API, troubleshooting y changelog.
- Mantener ADRs, diagramas Mermaid, runbooks, matriz de compatibilidad y guía de extensión.
- Timestamps UTC en persistencia; zona local solo en presentación.

11. Forma de trabajo con Codex

Antes de editar:

1. Leer este archivo, ADRs, contratos, migraciones, ejemplos y tests relacionados.

2. Resumir el estado actual y contradicciones.
3. Proponer un plan breve con archivos y riesgos.
4. Implementar código completo; evitar pseudocódigo y stubs silenciosos.
5. Ejecutar lint, type checking, tests y migraciones aplicables.
6. Corregir fallas antes de finalizar.
7. Entregar evidencia de comandos, archivos modificados, decisiones y limitaciones.

No modificar contratos públicos sin actualizar consumidores, esquemas, migraciones, documentación y tests. Los mocks se permiten en tests y simuladores, nunca para ocultar funcionalidad faltante en producción.

12. Definición de terminado

Una tarea está terminada cuando:

- Compila/inicia en la plataforma objetivo.
- Las migraciones aplican desde una base vacía y desde la versión anterior soportada.
- Tests y linters pasan.
- Hay criterios de aceptación verificables.
- Se documentaron configuración, seguridad, observabilidad, rollback y limitaciones.
- No se incluyeron secretos, binarios no verificados ni archivos temporales.
- El cambio es compatible con la estrategia de versionado y extensión.

Uso de los prompts con Codex - WiFi Test Orchestrator v2.0

1. Objetivo del paquete

Este paquete divide el desarrollo en fases pequeñas y auditables. Cada prompt debe ejecutarse desde la raíz del repositorio, con `AGENTS.md` presente. No ejecutar todos los prompts en una única sesión: cada fase debe producir código revisable, tests y documentación.

2. Preparación

1. Crear el repositorio dentro de WSL2 para obtener mejor rendimiento con Docker Desktop.
2. Copiar `00\_AGENTS\_master\_context.md` como `AGENTS.md` en la raíz.
3. Crear una rama principal protegida y una rama/worktree por fase cuando corresponda.
4. Instalar Git, Docker Desktop/WSL2, Python, Node.js, JDK/Android Studio y Xcode en un Mac para iOS.
5. Guardar secretos solo en `.env` no versionado o secret stores.

3. Orden recomendado

Ejecutar los prompts 01 a 26 en orden. No paralelizar antes de completar el Prompt 05 y estabilizar OpenAPI, JSON Schemas, test vectors y contratos de capabilities.

Después del Prompt 05 pueden desarrollarse en paralelo:

- Prompt 06: Windows.
- Prompt 07: Linux/Capture Node.
- Prompt 08: Android.
- Prompt 09: iOS.

Los Prompts 13, 14 y 15 pueden avanzar en paralelo después del Prompt 12, siempre que usen los mismos contratos del `TrafficGenerator`.

4. Ciclo de cada fase

```
git status
git add .
git commit -m "checkpoint: before phase XX"
```

Prompt previo para Codex:

Antes de implementar, inspecciona AGENTS.md, docs/architecture, ADRs, migraciones, OpenAPI, JSON Schemas, ejemplos y tests relacionados. Resumí el estado actual, detecta contradicciones y proponé un plan breve. No cambies contratos públicos sin una estrategia de compatibilidad.

Después de implementar:

Revisá el diff como reviewer senior. Buscá problemas de seguridad, concurrencia, idempotencia, migraciones, compatibilidad, secretos, lifecycle mobile, permisos, entitlements, supuestos Wi-Fi, generación de tráfico no acotada y tests insuficientes. Corregí los problemas y ejecutá nuevamente lint, type checking, tests, contract tests y migraciones.

```
git add .
git commit -m "feat: complete phase XX"
```

5. Evidencia obligatoria

Cada fase debe dejar en su handoff:

- Archivos creados/modificados.
- Decisiones y ADRs.
- Comandos ejecutados y resultado.
- Tests agregados.
- Métricas de cobertura relevantes.
- Riesgos, limitaciones y deuda técnica.
- Instrucciones de rollback.
- Impacto en contratos, migraciones y compatibilidad.

6. Reglas de trabajo paralelo

- Usar worktrees para agentes por plataforma.
- No editar simultáneamente el mismo schema desde dos ramas.
- Integrar primero contratos y test vectors, luego implementaciones.
- Rebasear y ejecutar toda la matriz de contract tests antes de mergear.

- En iOS se requiere un host macOS con Xcode; Codex en Windows no puede compilar ni firmar una app iOS localmente.

7. Archivos de ejemplo

La carpeta `examples/` contiene mensajes y manifests de referencia. Deben convertirse en golden fixtures usados por los contract tests de Python, TypeScript, Kotlin y Swift.

Changelog del paquete de prompts

2.0 - 2026-07-11

- Motor de tráfico desacoplado de iperf3.
- Prompts separados para arquitectura de tráfico, proveedor iperf3, tráfico HTTP/native y latency-under-load.
- Flent y Tcpreplay incorporados como extensiones especializadas y no como dependencias obligatorias.
- Nuevo modelo de perfiles de tráfico y pool de servidores de prueba.
- Guía de SDK de plugins, versionado, migraciones y compatibilidad hacia atrás.
- Mayor detalle de time synchronization, comparabilidad de resultados, seguridad, cuotas y runbooks.
- Fases ampliadas de 20 a 26.
- Documentación y troubleshooting como entregable de primera clase.

1.1 - 2026-07-11

- Agentes Windows, Linux, Android e iOS.
- Servidor Windows-first portable.
- Modelo capability-driven y restricciones mobile.

Prompt 01 - Arquitectura, ADRs y modelo de amenazas

Objetivo

Definir la arquitectura objetivo antes de escribir funcionalidades, incluyendo límites de dominio, deployment, seguridad, capabilities, extensibilidad y restricciones por plataforma.

Prerrequisitos

- Repositorio inicial con AGENTS.md.
- No se requiere código funcional todavía.

Alcance

- Diagramas de contexto, contenedores y componentes.
- ADRs para stack, contratos, cola, almacenamiento, mobile lifecycle, tráfico, captura y plugins.
- Threat model con activos, actores, fronteras de confianza y controles.
- Matriz de capacidades Windows/Linux/Android/iOS/Capture Node.
- Glosario técnico y lista de supuestos.

Requisitos de implementación

- Crear `docs/architecture/` con diagramas Mermaid versionados.
- Separar control plane, data plane, telemetry plane, artifact plane e integration plane.
- Definir topologías de laboratorio: servidor único, servidor + traffic nodes y deployment distribuido.
- Definir disponibilidad, escalabilidad inicial y límites del MVP.
- Definir modelo de capability con versión, condición, permisos, límites y reason.
- Definir estrategia de compatibilidad de contratos y plugin API.
- Registrar cada decisión relevante como ADR con status, contexto, decisión, consecuencias y alternativas.

Tests obligatorios

- Validar sintaxis Mermaid.
- Añadir lint/links check para documentación.
- Revisión de threat model contra casos de abuso de generación de tráfico, captura y credenciales.

Documentación obligatoria

- README de arquitectura.
- ADRs numerados.
- Matriz de capacidades y limitaciones.
- Diagrama de secuencia de enrolamiento y ejecución.

Criterios de aceptación

- La arquitectura puede explicar dónde vive cada responsabilidad.
- No existen dependencias funcionales del servidor respecto del host Windows.
- Las restricciones mobile están explícitas.
- Los futuros plugins pueden añadirse sin modificar el núcleo.

Forma de entrega

Antes de modificar código, inspeccioná `AGENTS.md`, ADRs, contratos, migraciones, ejemplos y tests relacionados. Presentá un plan breve. Implementá código completo, ejecutá las validaciones, corregí fallas y entregá un handoff con archivos, decisiones, comandos, resultados, riesgos y rollback.

Prompt 02 - Monorepo y servidor Windows-first portable

Objetivo

Crear el esqueleto ejecutable del monorepo y un entorno local reproducible en Windows 11 mediante Docker Desktop/WSL2.

Prerrequisitos

- Prompt 01 completado y ADRs aprobados.

Alcance

- Monorepo.
- Docker Compose.
- Backend mínimo.
- Frontend mínimo.
- PostgreSQL, Redis, reverse proxy y artifact store local.
- Scripts PowerShell y Bash.
- CI básica.

Requisitos de implementación

- Crear carpetas `backend`, `frontend`, `agents`, `mobile`, `shared`, `integrations`, `deployment`, `docs`, `tests` y `scripts`.
- Usar imágenes Linux OCI y volúmenes/rutas portables.
- Agregar `.env.example`, profiles de Compose, health checks y readiness.
- Agregar `Makefile` y scripts PowerShell equivalentes.
- Agregar endpoint `/health/live` y `/health/ready`.
- Crear agente simulado para smoke tests.
- No exponer Redis/PostgreSQL fuera de la red interna por defecto.
- Configurar logs estructurados y correlation IDs desde el inicio.

Tests obligatorios

- Smoke test de Compose desde entorno limpio.
- Health checks.
- Test de frontend que consume backend.
- CI en Linux para backend/frontend.
- Validar que no haya rutas absolutas de Windows en código.

Documentación obligatoria

- Guía Windows + WSL2.
- Guía alternativa Linux.
- Mapa del repositorio.
- Troubleshooting de puertos, certificados y volúmenes.

Criterios de aceptación

- `docker compose up` levanta todos los servicios.
- Un usuario puede abrir frontend y ver estado del sistema.
- Un agente simulado aparece en inventario.
- La CI pasa.

Forma de entrega

Antes de modificar código, inspeccioná `AGENTS.md`, ADRs, contratos, migraciones, ejemplos y tests relacionados. Presentá un plan breve. Implementá código completo, ejecutá las validaciones, corregí fallas y entregá un handoff con archivos, decisiones, comandos, resultados, riesgos y rollback.

Prompt 03 - Dominio backend, base de datos, autenticación y RBAC

Objetivo

Implementar el núcleo de dominio, persistencia, migraciones, usuarios, roles, sesiones y auditoría.

Prerrequisitos

- Prompt 02 funcional.
- PostgreSQL disponible.

Alcance

- Modelos y migraciones.
- Auth de usuarios.
- RBAC.
- Auditoría.
- Configuración y errores API.

Requisitos de implementación

- Modelar User, Role, Permission, Session/RefreshToken, Agent, Device, Capability, TestDefinition, Campaign, Execution, Metric, Artifact, AuditLog y EnrollmentToken.
- Usar UUID y timestamps UTC.
- Implementar access/refresh tokens con rotación y revocación.
- Hash seguro de contraseñas.
- Seeds y comando para crear administrador.
- Auditar login, cambios de rol, tokens, campañas y acciones sensibles.
- Errores estructurados con code, message, details y correlation_id.
- Índices y constraints explícitos.
- No almacenar secretos en texto plano.

Tests obligatorios

- Migración desde base vacía y downgrade controlado.
- Tests RBAC por endpoint.
- Rotación/revocación de refresh token.
- Auditoría.
- Constraints e idempotencia.

Documentación obligatoria

- ERD.
- Referencia de roles/permisos.
- Runbook de bootstrap admin.
- Política de retención de auditoría.

Criterios de aceptación

- Auth y RBAC protegen endpoints.
- Migraciones reproducibles.
- Auditoría identifica actor, acción, recurso y resultado.
- No se filtran secretos.

Forma de entrega

Antes de modificar código, inspeccioná `AGENTS.md`, ADRs, contratos, migraciones, ejemplos y tests relacionados. Presentá un plan breve. Implementá código completo, ejecutá las validaciones, corregí fallas y entregá un handoff con archivos, decisiones, comandos, resultados, riesgos y rollback.

Prompt 04 - Contratos, capabilities y enrolamiento de agentes

Objetivo

Estabilizar los contratos compartidos y el proceso seguro de enrolamiento, presencia, rotación y revocación.

Prerrequisitos

- Prompt 03 completo.
- ADRs de protocolo aprobados.

Alcance

- OpenAPI.
- JSON Schemas.
- Golden fixtures.
- Enrolamiento.
- Heartbeat/presence.
- Capability manifests.

Requisitos de implementación

- Definir schemas para AgentRegistration, AgentCredential, CapabilityManifest, TaskEnvelope, ProgressEvent, TestResult y ArtifactManifest.
- Incluir schema_version e idempotency_key.
- Tokens de enrolamiento de uso limitado y expiración.
- Emitir identidad individual y credencial rotatable.
- Soportar revocación inmediata.
- Heartbeat desktop; presence/fetch mobile.
- Capability manifest con supported/conditional/permission_required/user_interaction_required.
- Crear generadores/validadores de modelos para Python, TypeScript, Kotlin y Swift cuando sea viable.
- Mantener ejemplos en `shared/contracts/examples`.

Tests obligatorios

- Contract tests en cuatro lenguajes o validadores equivalentes.
- Replay/idempotency.
- Token expirado/consumido.
- Clock skew.
- Revocación.
- Manifests incompatibles.

Documentación obligatoria

- Referencia del protocolo.
- Diagramas de secuencia.
- Catálogo de capabilities.
- Política de versionado.

Criterios de aceptación

- Un agente simulado se enrola y rota credenciales.
- Un agente revocado no obtiene tareas ni sube resultados.
- Los fixtures son válidos en todos los consumidores.
- Cambios incompatibles son detectados.

Forma de entrega

Antes de modificar código, inspeccioná `AGENTS.md`, ADRs, contratos, migraciones, ejemplos y tests relacionados. Presentá un plan breve. Implementá código completo, ejecutá las validaciones, corregí fallas y entregá un handoff con archivos, decisiones, comandos, resultados, riesgos y rollback.

Prompt 05 - Núcleo compartido de agentes desktop

Objetivo

Implementar el runtime común para Windows y Linux sin acoplarlo a comandos o APIs de un sistema operativo.

Prerrequisitos

- Prompt 04 completo.
- Contratos publicados.

Alcance

- Transporte HTTPS.
- Cola SQLite.
- Task runner.
- Plugin registry.
- Adaptadores de plataforma.
- Configuración y diagnóstico.

Requisitos de implementación

- Crear interfaces PlatformAdapter, WifiCollector, NetworkController, ProcessRunner, SecretStore y ServiceManager.
- Persistir tareas, resultados y uploads pendientes en SQLite.
- Garantizar exactly-once effect mediante idempotencia aunque la entrega sea at-least-once.
- Backoff con jitter.
- Allowlist de plugins y parámetros.
- Timeout, cancelación y cleanup.
- CLI: enroll, run, status, doctor, capabilities.
- Logs estructurados con redacción de secretos.
- No importar módulos Windows desde Linux ni viceversa.
- Proveer simuladores para CI.

Tests obligatorios

- Reinicio durante tarea.
- Servidor offline.
- Resultado duplicado.
- Cancelación.
- Corrupción parcial de cola.
- Plugin no autorizado.
- Property tests de estados.

Documentación obligatoria

- Arquitectura del agente.
- Formato de configuración.
- Guía de desarrollo de plugins desktop.
- Troubleshooting doctor.

Criterios de aceptación

- El núcleo pasa tests en Windows y Linux.
- No duplica efectos.
- Recupera cola después de reinicio.
- No ejecuta comandos arbitrarios.

Forma de entrega

Antes de modificar código, inspecciona `AGENTS.md`, ADRs, contratos, migraciones, ejemplos y tests relacionados. Presenta un plan breve. Implementa código completo, ejecuta las validaciones, corrige fallas y entregará un handoff con archivos, decisiones, comandos, resultados, riesgos y rollback.

Prompt 06 - Agente Windows

Objetivo

Implementar el adaptador Windows y empaquetarlo como servicio instalable para Windows 10/11.

Prerrequisitos

- Prompt 05 completo.
- Equipo/VM Windows para integración.

Alcance

- Inventario.
- Telemetría Wi-Fi.
- Control permitido.
- Servicio.
- Packaging preliminar.
- Captura IP opcional.

Requisitos de implementación

- Priorizar Windows Native Wi-Fi API y APIs estructuradas; PowerShell como soporte; netsh solo fallback localizado.
- Detectar interfaces, driver, SSID/BSSID, señal, canal/frecuencia cuando estén disponibles, TX/RX rate, IP, gateway, DNS y counters.
- Registrar source/availability por campo.
- Implementar Windows Service con recuperación.
- Usar DPAPI/Windows Credential Manager para secretos.
- Integrar iperf3 y dumpcap mediante ProcessRunner allowlisted, no desde lógica de dominio.
- Mapear interfaces Npcap de forma robusta.
- Agregar instalador firmable y uninstall limpio.
- No asumir privilegios administrativos permanentes.

Tests obligatorios

- Fixtures de salida localizada.
- Windows Service lifecycle.
- Permisos insuficientes.
- Adaptadores múltiples.
- Sleep/resume y cambio de red.
- Integración real en Windows 10/11.

Documentación obligatoria

- Matriz de campos Windows.
- Instalación, upgrade y rollback.
- Requisitos de Npcap/iperf3.
- Troubleshooting de idioma y permisos.

Criterios de aceptación

- Se enrola como servicio.
- Publica datos honestos.
- Ejecuta pruebas permitidas.
- Sobrevive a reboot.
- Desinstala sin dejar secretos.

Forma de entrega

Antes de modificar código, inspecciona `AGENTS.md`, ADRs, contratos, migraciones, ejemplos y tests relacionados. Presenta un plan breve. Implementa código completo, ejecuta las validaciones, corrige fallas y entregará un handoff con archivos, decisiones, comandos, resultados, riesgos y rollback.

Prompt 07 - Agente Linux y Capture Node

Objetivo

Implementar el agente Linux y un rol especializado de Capture Node para captura 802.11 y herramientas avanzadas.

Prerrequisitos

- Prompt 05 completo.
- Host Linux/VM para integración.

Alcance

- NetworkManager/DBus.
- iw/ip/ethtool fallbacks.
- systemd.
- Packaging.
- Capture Node.
- Flent/Tcpreplay opcionales.

Requisitos de implementación

- Usar APIs estructuradas cuando existan y encapsular comandos.
- Detectar interfaces, wiphy, driver, SSID/BSSID, frecuencia/canal, ancho, RSSI, rates, IP y counters.
- Servicio systemd con hardening.
- Secretos mediante permisos estrictos o keyring.
- Paquetes DEB primero; diseño para RPM.
- Rol Capture Node separado con NICs compatibles, channel lock, radiotap, timestamp y artifact upload.
- Flent solo si dependencias están instaladas y capability declarada.
- Tcpreplay solo en Capture/Lab Node, destino allowlisted y namespace/red aislada.
- Nunca habilitar monitor mode automáticamente en un endpoint de usuario.

Tests obligatorios

- NetworkManager presente/ausente.
- systemd lifecycle.
- Cambio de interfaz.
- Permisos dumpcap.
- Capture Node simulation.
- Safety tests de tcpreplay.

Documentación obligatoria

- Matriz de distros.
- Instalación DEB.
- Configuración de capabilities especiales.
- Runbook de Capture Node y seguridad.

Criterios de aceptación

- Agente Linux funcional.
- Capture Node es un rol explícito.
- Herramientas avanzadas no están disponibles sin política.
- Rollback restaura interfaz.

Forma de entrega

Antes de modificar código, inspeccioná `AGENTS.md`, ADRs, contratos, migraciones, ejemplos y tests relacionados. Presentá un plan breve. Implementá código completo, ejecutá las validaciones, corregí fallas y entregá un handoff con archivos, decisiones, comandos, resultados, riesgos y rollback.

Prompt 08 - Agente Android nativo

Objetivo

Crear una aplicación Android nativa que ejecute pruebas de red de forma honesta y compatible con restricciones modernas del sistema.

Prerrequisitos

- Prompt 04 completo.
- Android Studio/JDK disponibles.

Alcance

- Enrolamiento.
- Capabilities.
- Room queue.
- UI de pruebas.
- Foreground execution.
- WorkManager sync.
- Probes nativos.

Requisitos de implementación

- Kotlin + Jetpack Compose o UI aprobada.
- Solicitar permisos mínimos y explicar su uso.
- Compatibilidad por API level; NEARBY_WIFI_DEVICES y permisos legacy según corresponda.
- No asumir scans ilimitados ni conexión silenciosa a redes.
- Foreground Service solo cuando sea válido y visible; manejar límites/timeouts por versión.
- WorkManager para sincronización diferible, no para ejecución exacta.
- OkHttp para HTTP; sockets nativos para TCP/UDP; ConnectivityManager/NetworkCallback para path.
- Room para cola offline.
- Android Keystore para secretos.
- Capturar contexto térmico/batería solo con APIs permitidas.
- Mostrar claramente WAITING_FOR_USER/FOREGROUND_REQUIRED.

Tests obligatorios

- Unit tests.
- Instrumented tests en al menos dos API levels.
- Permiso denegado.
- App background/foreground.
- Cambio Wi-Fi/celular.
- Doze/force-stop documentado.
- Proceso terminado durante prueba.

Documentación obligatoria

- Matriz por API level.
- Permisos y privacy.
- Guía de ejecución foreground.
- Limitaciones de scan y background.

Criterios de aceptación

- La app no promete daemon permanente.
- Reanuda uploads sin duplicar.
- Explica tareas bloqueadas.
- Probes devuelven método y disponibilidad.

Forma de entrega

Antes de modificar código, inspecciona `AGENTS.md`, ADRs, contratos, migraciones, ejemplos y tests relacionados. Presenta un plan breve. Implementa código completo, ejecuta las validaciones, corrige fallas y entrega un handoff con archivos, decisiones, comandos, resultados, riesgos y rollback.

Prompt 09 - Agente iOS/iPadOS nativo

Objetivo

Crear una aplicación Swift foreground-first para pruebas de red, sin depender de capacidades no expuestas por iOS.

Prerrequisitos

- Prompt 04 completo.
- Mac con Xcode y dispositivo real para integración.

Alcance

- Enrolamiento.
- Keychain.
- Capabilities.
- Cola local.
- Network.framework.
- URLSession.
- BackgroundTasks como optimización.
- UI de campañas.

Requisitos de implementación

- Swift/SwiftUI.
- URLSession para HTTP y uploads/downloads.
- Network.framework para TCP/UDP/path monitoring.
- Keychain para credenciales.
- Persistencia local con Core Data/SQLite o solución aprobada.
- BGTaskScheduler solo para sincronización elegible; no garantizar horario.
- APNs como hint, no orden garantizada.
- Capability `foreground\_required` para pruebas largas.
- Access WiFi Information solo si entitlement y condiciones aplican.
- No prometer RSSI, scan genérico, monitor mode ni captura general.
- Preparar integración MDM/TestFlight sin asumir App Store público.
- Registrar thermal state y battery state solo cuando sea permitido.

Tests obligatorios

- XCTest.
- Network.framework mocks.
- Foreground/background.
- App terminada por usuario.
- Sin entitlement.
- Cambio de path.
- Device real.

Documentación obligatoria

- Matriz iOS/iPadOS.
- Entitlements.
- Distribución.
- Limitaciones y semántica de background.

Criterios de aceptación

- La app ejecuta pruebas foreground.
- No reporta datos inexistentes.
- Resultados se sincronizan al reabrir.
- Tareas incompatibles quedan BLOCKED/SKIPPED.

Forma de entrega

Antes de modificar código, inspecciona `AGENTS.md`, ADRs, contratos, migraciones, ejemplos y tests relacionados. Presenta un plan breve. Implementa código completo, ejecuta las validaciones, corrige fallas y entrega un handoff con archivos, decisiones, comandos, resultados, riesgos y rollback.

Prompt 10 - Orquestación, leases y entrega mobile

Objetivo

Implementar el ciclo de vida de tareas para desktop y mobile con idempotencia, expiración, recuperación y estados explícitos.

Prerrequisitos

- Prompts 04-09 disponibles.

Alcance

- Queueing.
- Leases.
- Claims.
- Progress.
- Cancellation.
- Mobile hints.
- Preflight de capabilities.
- Concurrency.

Requisitos de implementación

- Estados: QUEUED, OFFERED, CLAIMED, RUNNING, WAITING_FOR_FOREGROUND, WAITING_FOR_USER, COMPLETED, FAILED, CANCELED, EXPIRED, SKIPPED, BLOCKED.
- Lease con renovación y fencing token.
- No reasignar efecto activo sin expiración segura.
- Push Android/iOS solo como señal.
- Preflight de capabilities, permisos, batería, conectividad y foreground.
- Límites de concurrencia global, por agente, test server y laboratorio.
- Scheduling con not_before/expires_at.
- Cancelación cooperativa y cleanup.
- Eventos inmutables y proyección de estado.
- WebSocket/SSE para frontend sin convertirlo en canal de control del agente.

Tests obligatorios

- Dos agentes reclaman la misma tarea.
- Lease expira.
- Progress fuera de orden.
- Resultado duplicado.
- Push no entregado.
- App cerrada.
- Cancelación durante upload.
- Chaos tests.

Documentación obligatoria

- Máquina de estados.
- Runbook de tareas atascadas.
- Semántica de mobile delivery.
- Métricas operativas.

Criterios de aceptación

- No hay doble ejecución efectiva.
- Estados explican por qué no se ejecutó.
- Desktop recupera automáticamente.
- Mobile no depende de ejecución inmediata.

Forma de entrega

Antes de modificar código, inspecciona `AGENTS.md`, ADRs, contratos, migraciones, ejemplos y tests relacionados. Presenta un plan breve. Implementa código completo, ejecuta las validaciones, corrige fallas y entrega un handoff con archivos, decisiones, comandos, resultados, riesgos y rollback.

Prompt 11 - Pruebas básicas de red y conectividad

Objetivo

Implementar plugins portables de latencia, DNS, HTTP y conectividad por etapas.

Prerrequisitos

- Prompt 10 completo.
- Plugin API disponible.

Alcance

- Latency probe.
- DNS.
- HTTP/HTTPS.
- Connectivity pipeline.
- Stability session básica.

Requisitos de implementación

- LatencyProbe abstracto con ICMP, TCP connect, HTTP request y UDP echo cuando corresponda.
- No comparar métodos distintos sin etiquetarlos.
- DNS usando resolver del SO y opción de servidor dirigido donde la plataforma lo permita.
- HTTP con DNS, connect, TLS, TTFB, total, status y bytes según API disponible.
- Connectivity pipeline: interface/path, IP, gateway cuando sea observable, DNS y application endpoint.
- Stability session con muestras periódicas, desconexiones, BSSID/path changes y gaps.
- Thresholds versionados y snapshot.
- Destinos allowlisted.
- Timeouts y límites.

Tests obligatorios

- Éxito/falla por etapa.
- IPv4/IPv6.
- DNS timeout.
- TLS error.
- Redirects.
- Captive portal.
- Cambio de path.
- Comparabilidad.

Documentación obligatoria

- Referencia de cada método.
- Ejemplos de thresholds.
- Limitaciones por plataforma.
- Troubleshooting de resultados.

Criterios de aceptación

- El resultado identifica la etapa exacta.
- Conserva método y proveedor.
- No usa cero para datos faltantes.
- Los plugins son extensibles.

Forma de entrega

Antes de modificar código, inspeccioná `AGENTS.md`, ADRs, contratos, migraciones, ejemplos y tests relacionados. Presentá un plan breve. Implementá código completo, ejecutá las validaciones, corregí fallas y entregá un handoff con archivos, decisiones, comandos, resultados, riesgos y rollback.

Prompt 12 - Arquitectura del motor de tráfico y perfiles versionados

Objetivo

Diseñar e implementar el contrato central de generación de tráfico sin acoplar el dominio a iperf3, HTTP o una plataforma.

Prerrequisitos

- Prompts 04, 10 y 11 completos.

Alcance

- TrafficGenerator API.
- TrafficProfile.
- Server capability/reservation.
- Normalized results.
- Policy engine.
- Provider registry.

Requisitos de implementación

- Definir `TrafficGenerator.prepare/start/progress/stop/collect/cleanup` y semántica de cancelación.
- Definir protocolos TCP, UDP, HTTP y futuros QUIC/WebRTC sin implementarlos todos.
- Direcciones uplink, downlink, bidirectional y alternating.
- Parámetros: duration, bytes, streams, pacing, target bitrate, packet size, DSCP, IPv4/IPv6, warmup/cooldown.
- TrafficProfile versionado y reusable.
- TrafficServer pool con capabilities, capacity, health, reservations y affinity.
- Políticas de destino, puertos, duración, bitrate, concurrencia y horario.
- Normalized interval metrics con timestamps monotónicos y wall clock.
- Declarar provider/version/platform/method.
- Registrar raw result como artifact y resumen normalizado.
- Diseñar capability negotiation y fallback explícito, nunca silencioso.
- Soportar dry-run/preflight.

Tests obligatorios

- Schema/contract tests.
- Provider incompatible.
- Fallback explícito.
- Cancelación.
- Límites violados.
- Reserva concurrente.
- Intervalos fuera de orden.
- Golden fixtures.

Documentación obligatoria

- SDK TrafficGenerator.
- Schema TrafficProfile.
- Guía para agregar proveedor.
- Catálogo de perfiles recomendados.
- Políticas de seguridad.

Criterios de aceptación

- El dominio puede usar varios proveedores.
- Un perfil es reproducible e inmutable por versión.
- Los resultados son comparables solo cuando corresponde.
- Los límites bloquean abuso.

Forma de entrega

Antes de modificar código, inspeccioná `AGENTS.md`, ADRs, contratos, migraciones, ejemplos y tests relacionados. Presentá un plan breve. Implementá código completo, ejecutá las validaciones, corregí fallas y entregá un handoff con archivos, decisiones, comandos, resultados, riesgos y rollback.

Prompt 13 - Proveedor iperf3 y pool de servidores de prueba

Objetivo

Implementar iperf3 como proveedor principal de throughput para Windows/Linux y administrar servidores de prueba controlados.

Prerrequisitos

- Prompt 12 completo.
- iperf3 instalado o empaquetado de forma verificable.

Alcance

- Cliente iperf3.
- Servidor/nodo iperf3.
- TCP/UDP.
- Reverse/bidir.
- JSON/streaming.
- Reservations.
- Health.

Requisitos de implementación

- Ejecutar iperf3 mediante ProcessRunner allowlisted.
- Detectar versión y capacidades.
- Soportar TCP y UDP, uplink, reverse, bidirectional cuando la versión lo permita, streams paralelos, duration y bitrate.
- Usar JSON o json-stream; conservar salida cruda.
- Parsear intervals, end summary, retransmissions, jitter y loss según protocolo.
- Kill process tree y cleanup de puertos.
- Pool de servidores con tokens/reservas, max bitrate, max tests y server-bitrate-limit cuando esté disponible.
- No usar servidores públicos por defecto.
- Verificar hash/firma de binarios distribuidos y licencia.
- Capturar CPU y counters opcionales sin confundirlos con throughput.
- Fallback a provider alternativo debe ser explícito y registrado.

Tests obligatorios

- Versiones soportadas.
- JSON parcial/corrupto.
- Server busy.
- UDP loss.
- Cancelación.
- Process hang.
- Port collision.
- Bidir/reverse.
- Integration Windows/Linux.

Documentación obligatoria

- Instalación de iperf3.
- Operación del pool.
- Capacidades por versión.
- Troubleshooting y códigos de error.

Criterios de aceptación

- TCP/UDP funcionan contra nodo controlado.
- Intervals llegan en tiempo real.
- No quedan procesos/puertos huérfanos.
- Límites del pool se respetan.

Forma de entrega

Antes de modificar código, inspeccioná `AGENTS.md`, ADRs, contratos, migraciones, ejemplos y tests relacionados. Presentá un plan breve. Implementá código completo, ejecutá las validaciones, corregí fallas y entregá un handoff con archivos, decisiones, comandos, resultados, riesgos y rollback.

Prompt 14 - Tráfico HTTP y sockets nativos multiplataforma

Objetivo

Implementar proveedores de tráfico realista y portables para Windows, Linux, Android e iOS.

Prerrequisitos

- Prompt 12 completo.
- Endpoints de prueba controlados.

Alcance

- HTTP download/upload.
- Native TCP stream.
- UDP stream opcional.
- Mobile-friendly execution.
- Server endpoints.

Requisitos de implementación

- HTTP provider con archivos/payloads controlados, no cacheables, checksum y límites.
- Medir DNS/connect/TLS/TTFB/total cuando la API lo exponga.
- Soportar upload/download, fixed bytes y fixed duration.
- Sockets nativos con framing, sequence numbers y acknowledgements para medir bytes/intervalos sin mezclar con iperf3.
- Server de eco/throughput detrás de autenticación y reservas.
- Android: OkHttp + sockets; iOS: URLSession + Network.framework.
- Desktop: mismo contrato como fallback.
- Controlar network binding en mobile cuando sea permitido y reportarlo.
- Registrar thermal/battery/network path.
- Manejar app suspendida como interrupción explícita.
- No declarar equivalencia automática con iperf3.

Tests obligatorios

- Cache prevention.
- TLS.
- Partial upload.
- Cambio de red.
- App background.
- Checksum.
- IPv6.
- Servidor lento.
- Comparación cross-platform.

Documentación obligatoria

- Protocolo del servidor nativo.
- Métricas disponibles por plataforma.
- Perfiles mobile.
- Limitaciones de comparación.

Criterios de aceptación

- Android/iOS generan tráfico sin binario externo.
- Resultados incluyen método.
- El servidor impone cuotas.
- No hay falsa equivalencia con iperf3.

Forma de entrega

Antes de modificar código, inspecciona `AGENTS.md`, ADRs, contratos, migraciones, ejemplos y tests relacionados. Presenta un plan breve. Implementa código completo, ejecuta las validaciones, corrige fallas y entrega un handoff con archivos, decisiones, comandos, resultados, riesgos y rollback.

Prompt 15 - Latencia bajo carga, bufferbloat y Flent

Objetivo

Implementar pruebas compuestas que midan latencia antes, durante y después de una carga controlada, con una integración opcional de Flent.

Prerrequisitos

- Prompts 11-14 completos.

Alcance

- Composite test runner.
- Independent probes.
- Load phases.
- Bufferbloat metrics.
- Flent adapter opcional.
- RRUL-like profile.

Requisitos de implementación

- Ejecutar baseline, warmup, load, cooldown y recovery.
- Usar probe independiente simultáneo al TrafficGenerator.
- Soportar ICMP/TCP/HTTP latency con etiqueta de método.
- Calcular median, p95, p99, jitter, loss y latency increase respecto de baseline.
- Definir severidad configurable, no universal.
- Crear perfiles download-load, upload-load, bidirectional-load y multi-stream.
- Integrar Flent en Linux si está instalado; importar JSON y metadata.
- No hacer Flent dependencia del MVP ni ejecutar netperf remoto sin allowlist.
- Sincronizar timestamps monotónicos y conservar raw series.
- Visualizar series y fases.
- Distinguir bufferbloat inferido de causas Wi-Fi/radio no observadas.

Tests obligatorios

- Probe falla durante carga.
- Clock alignment.
- Baseline inestable.
- High loss.
- Cancelación.
- Flent ausente.
- Golden series.
- Regression test de cálculos.

Documentación obligatoria

- Definición de métricas.
- Perfiles y thresholds.
- Guía Flent.
- Interpretación y límites diagnósticos.

Criterios de aceptación

- La prueba muestra latencia en reposo y bajo carga.
- Puede ejecutarse sin Flent.
- Los cálculos son reproducibles.
- El reporte evita conclusiones no sustentadas.

Forma de entrega

Antes de modificar código, inspeccioná `AGENTS.md`, ADRs, contratos, migraciones, ejemplos y tests relacionados. Presentá un plan breve. Implementá código completo, ejecutá las validaciones, corregí fallas y entregá un handoff con archivos, decisiones, comandos, resultados, riesgos y rollback.

Prompt 16 - Telemetría, eventos Wi-Fi y sincronización temporal

Objetivo

Implementar ingestión escalable, availability maps, detección de eventos y una estrategia explícita de tiempo.

Prerrequisitos

- Agentes y contratos disponibles.

Alcance

- Samples.
- Batches.
- Events.
- Clock metadata.
- Retention.
- Downsampling.
- Exports.

Requisitos de implementación

- Batches idempotentes y compresión.
- Conservar collected_at, received_at, monotonic offset, clock quality y timezone metadata.
- No confiar ciegamente en reloj del agente; detectar skew y jumps.
- Normalizar Wi-Fi fields con source/availability/confidence.
- Eventos: association, disconnect, BSSID change, channel/frequency change, IP/path change, agent restart y telemetry gap.
- Reglas de deduplicación y debounce.
- Retención configurable, particionamiento y downsampling.
- Export CSV/JSON.
- Métricas de ingest lag, rejected samples y gaps.
- Preparar integración futura con TimescaleDB sin hacerla obligatoria.

Tests obligatorios

- Batch duplicado.
- Out-of-order.
- Clock jump.
- Offline backlog.
- Event debounce.
- Retention.
- Load tests.

Documentación obligatoria

- Diccionario de datos.
- Semántica temporal.
- Política de retención.
- Guía de queries.

Criterios de aceptación

- No duplica muestras.
- Eventos son explicables.
- Series preservan calidad temporal.
- Dashboards funcionan con downsampling.

Forma de entrega

Antes de modificar código, inspecciona `AGENTS.md`, ADRs, contratos, migraciones, ejemplos y tests relacionados. Presenta un plan breve. Implementa código completo, ejecuta las validaciones, corrige fallas y entrega un handoff con archivos, decisiones, comandos, resultados, riesgos y rollback.

Prompt 17 - Casos, suites, planes, campañas y comparación

Objetivo

Implementar la capa de gestión de pruebas con versionado, snapshots, scheduling y comparación honesta.

Prerrequisitos

- Prompts 10-16 completos.

Alcance

- TestCase.
- Suite.
- Plan.
- Campaign.
- Targets.
- Thresholds.
- Baseline.
- Comparison.

Requisitos de implementación

- Versionar definiciones; published versions inmutables.
- Snapshot de plugins, providers, manifests, thresholds y parámetros al lanzar.
- Selección por dispositivo, grupo, tag, plataforma o capability.
- Preflight que muestre ejecutable/skipped/blocked.
- Repeticiones, serial/paralelo, max concurrency y ventanas.
- Pausa/reanudación/cancelación.
- Rerun selectivo.
- Baselines por modelo/firmware/plataforma.
- Comparison engine con reglas de equivalencia de método.
- Estadística básica con sample size, dispersion y outliers documentados.
- No cambiar PASS/FAIL histórico al editar thresholds.

Tests obligatorios

- Version publish.
- Snapshot.
- Campaña heterogénea.
- Rerun.
- Comparison incompatible.
- Scheduling.
- Concurrency.
- Historical immutability.

Documentación obligatoria

- Modelo funcional.
- Guía de diseño de campañas.
- Reglas de comparación.
- Ejemplos de ATP.

Criterios de aceptación

- Campañas reproducibles.
- Resultados históricos inmutables.
- Preflight evita fallas por incompatibilidad.
- Comparaciones muestran método y tamaño de muestra.

Forma de entrega

Antes de modificar código, inspecciona `AGENTS.md`, ADRs, contratos, migraciones, ejemplos y tests relacionados. Presenta un plan breve. Implementa código completo, ejecuta las validaciones, corrige fallas y entrega un handoff con archivos, decisiones, comandos, resultados, riesgos y rollback.

Prompt 18 - Frontend, dashboards y flujos operativos

Objetivo

Construir una interfaz web que permita operar el producto sin depender de Swagger o acceso directo a base de datos.

Prerrequisitos

- Backend y campañas disponibles.

Alcance

- Login.
- Inventory.
- Capabilities.
- Campaign builder.
- Live progress.
- Results.
- Telemetry.
- Artifacts.
- Admin.

Requisitos de implementación

- React/TypeScript con API client generado o tipado.
- Pantallas responsive y accesibles.
- Inventario con estado, last seen, plataforma, versión y capabilities.
- Detalle con availability/source por campo.
- Campaign builder con preflight y advertencias mobile.
- Progreso por eventos.
- Gráficos de throughput, latency-under-load, RSSI, PHY rate y loss.
- Comparador con filtros por método/plataforma/firmware.
- Descarga de artefactos y reportes.
- RBAC en UI y backend.
- Timezone configurable.
- Estados vacíos/errores claros.
- No ocultar SKIPPED/BLOCKED.

Tests obligatorios

- Component tests.
- API contract tests.
- Playwright E2E.
- Accessibility checks.
- Large dataset pagination.
- Reconnect live updates.

Documentación obligatoria

- Guía de usuario.
- Mapa de pantallas.
- Convenciones de gráficos.
- Troubleshooting UI.

Criterios de aceptación

- Operador completa flujo end-to-end.
- Los gráficos explican método.
- RBAC efectivo.
- Errores y limitaciones visibles.

Forma de entrega

Antes de modificar código, inspecciona `AGENTS.md`, ADRs, contratos, migraciones, ejemplos y tests relacionados. Presenta un plan breve. Implementa código completo, ejecuta las validaciones, corrige fallas y entrega un handoff con archivos, decisiones, comandos, resultados, riesgos y rollback.

Prompt 19 - Artefactos, captura IP y reportes

Objetivo

Implementar almacenamiento seguro de artefactos, captura permitida por plataforma y reportes trazables.

Prerrequisitos

- Prompts 06-18 completos.

Alcance

- Artifact store.
- Multipart upload.
- Hashes.
- Capture providers.
- Report renderers.
- Retention.

Requisitos de implementación

- Artifact manifest con type, media_type, size, sha256, source, execution y retention class.
- Upload resumable y cuotas.
- Filesystem local y backend S3/MinIO.
- Windows: Npcap/dumpcap opcional; Linux: dumpcap/tcpdump; Android VpnService solo fase opcional con consentimiento; iOS sin captura general por defecto.
- CaptureProvider con filtros, interface mapping, duration y max size.
- PCAP analysis básico con tshark en worker aislado.
- Reportes HTML/PDF/CSV/JSON con snapshot, plataforma, methods, availability, thresholds y motivos.
- Redacción de secretos/PII.
- Signed URLs o autorización por descarga.
- Expiración y legal hold opcional.
- No incrustar PCAP completo en PDF.

Tests obligatorios

- Upload interrumpido.
- Hash mismatch.
- Quota.
- Unauthorized download.
- Capture timeout.
- Report deterministic.
- Retention deletion.
- Malicious filename.

Documentación obligatoria

- Catálogo de artefactos.
- Política de retención.
- Guía de captura.
- Template de reporte y limitaciones.

Criterios de aceptación

- Artefactos íntegros.
- Captura acotada.
- Reportes reproducibles.
- Acceso auditado.

Forma de entrega

Antes de modificar código, inspeccioná `AGENTS.md`, ADRs, contratos, migraciones, ejemplos y tests relacionados. Presentá un plan breve. Implementá código completo, ejecutá las validaciones, corregí fallas y entregá un handoff con archivos, decisiones, comandos, resultados, riesgos y rollback.

Prompt 20 - Replay de PCAP y nodos especializados de laboratorio

Objetivo

Diseñar la incorporación segura de Tcpreplay y otras herramientas de alto privilegio sin exponerlas a agentes comunes.

Prerrequisitos

- Prompt 19 completo.
- Capture/Lab Node Linux definido.

Alcance

- LabNode role.
- ReplayProfile.
- Tcpreplay provider.
- Network isolation.
- Safety approvals.
- Results/artifacts.

Requisitos de implementación

- Crear rol `LabNode` separado con enrolamiento y políticas específicas.
- Tcpreplay solo en Linux y redes de laboratorio identificadas.
- ReplayProfile versionado con PCAP hash, interface, rate, loops, rewrite policy y destination scope.
- Requerir aprobación/permiso adicional y preflight de interfaz/red.
- Usar namespaces, VLAN/VRF o aislamiento disponible.
- Prohibir PCAP no autorizado, destinos públicos y replay indefinido.
- Validar tamaño, tipo y hash.
- Capturar counters de envío, drops y timestamps.
- Soportar dry-run y simulador.
- Documentar que replay de paquetes no equivale a tráfico de aplicación con estado.
- Preparar hooks futuros para TRex u otros generadores, sin implementarlos.

Tests obligatorios

- Policy denial.
- Wrong interface.
- Public route detection.
- Timeout.
- PCAP corrupto.
- Rate limit.
- Cleanup.
- Audit.

Documentación obligatoria

- Runbook de LabNode.
- Threat model específico.
- Guía de ReplayProfile.
- Procedimiento de aprobación.

Criterios de aceptación

- Un endpoint común no puede ejecutar replay.
- El LabNode requiere política explícita.
- No queda tráfico tras cancelación.
- La auditoría es completa.

Forma de entrega

Antes de modificar código, inspeccioná `AGENTS.md`, ADRs, contratos, migraciones, ejemplos y tests relacionados. Presentá un plan breve. Implementá código completo, ejecutá las validaciones, corregí fallas y entregá un handoff con archivos, decisiones, comandos, resultados, riesgos y rollback.

Prompt 21 - Adaptadores de AP, gateways, ONTs y controladores

Objetivo

Crear un framework de integraciones externas para correlacionar estado de infraestructura y aplicar configuraciones controladas.

Prerrequisitos

- Dominio y contratos estables.

Alcance

- Adapter SDK.
- Read operations.
- Optional write operations.
- Secrets.
- Polling/events.
- Correlation.

Requisitos de implementación

- Interfaces `get_device_info`, `get_radios`, `get_clients`, `get_channel`, `get_width`, `get_utilization`, `get_noise`, `get_events` y `get_client_metrics`.
- Adaptadores iniciales mock HTTP y SNMP simulado.
- Credenciales en secret store.
- Timeout/retry/circuit breaker.
- Mapping de MAC/agent/device con confianza.
- Write actions separadas, RBAC fuerte, snapshot before/after, validate y rollback.
- No ejecutar SSH genérico como consola remota.
- Preparar TR-181/USP, REST, SNMP y controller APIs.
- Registrar source, timestamp y freshness.
- Rate limits por sistema externo.

Tests obligatorios

- Adapter unavailable.
- Stale data.
- Credential failure.
- Mapping ambiguity.
- Rollback.
- Circuit breaker.
- Contract tests.

Documentación obligatoria

- SDK de adaptadores.
- Guía de credenciales.
- Ejemplo mock.
- Matriz de datos por proveedor.

Criterios de aceptación

- Datos externos se distinguen de datos del agente.
- Write requiere permisos y rollback.
- Un nuevo adaptador no modifica el núcleo.

Forma de entrega

Antes de modificar código, inspecciona `AGENTS.md`, ADRs, contratos, migraciones, ejemplos y tests relacionados. Presenta un plan breve. Implementa código completo, ejecuta las validaciones, corrige fallas y entrega un handoff con archivos, decisiones, comandos, resultados, riesgos y rollback.

Prompt 22 - SDK de plugins, versionado y migraciones

Objetivo

Convertir la extensibilidad en un contrato formal para futuras funciones sin romper instalaciones existentes.

Prerrequisitos

- Arquitectura y varios plugins existentes.

Alcance

- Plugin manifests.
- Lifecycle.
- Compatibility.
- Schema migrations.
- Feature flags.
- Deprecation.
- Examples.

Requisitos de implementación

- PluginManifest con id, semantic version, API version, capabilities, config schema, result schema, platforms, permissions, resources y signatures.
- Discovery controlado; no cargar código arbitrario descargado.
- Version negotiation entre servidor/agente/plugin.
- Configuraciones validadas y secrets references.
- Result schema versionado.
- Migraciones de DB y de configuración con rollback.
- Feature flags y staged rollout.
- Deprecation policy con warning, sunset y compatibility window.
- SDK examples para TestPlugin, TrafficGenerator, Adapter y ReportRenderer.
- Conformance test suite para terceros internos.
- Changelog y release notes obligatorios.
- No permitir plugins sin firma/allowlist en producción.

Tests obligatorios

- Plugin incompatible.
- Migration upgrade/downgrade.
- Old agent/new server.
- New agent/old server.
- Feature flag rollback.
- Conformance tests.
- Signature failure.

Documentación obligatoria

- Guía de extensión paso a paso.
- Política semver.
- Matriz de compatibilidad.
- Plantillas de ADR/migration/changelog.

Criterios de aceptación

- Se añade un plugin de ejemplo sin tocar el core.
- Compatibilidad se valida automáticamente.
- Migraciones son reversibles o documentan no-retorno.
- Deprecaciones son visibles.

Forma de entrega

Antes de modificar código, inspecciona `AGENTS.md`, ADRs, contratos, migraciones, ejemplos y tests relacionados. Presenta un plan breve. Implementa código completo, ejecuta las validaciones, corrige fallas y entrega un handoff con archivos, decisiones, comandos, resultados, riesgos y rollback.

Prompt 23 - Seguridad, observabilidad y CI/CD

Objetivo

Endurecer el sistema, instrumentarlo y crear pipelines reproducibles para todas las plataformas.

Prerrequisitos

- Funcionalidad principal integrada.

Alcance

- Security controls.
- Metrics/logs/traces.
- CI matrices.
- SAST/SCA.
- SBOM.
- Load/chaos.
- Release gates.

Requisitos de implementación

- Threat model actualizado.
- TLS/mTLS o credenciales robustas según ADR.
- Rate limiting y abuse controls.
- Secret scanning, dependency scanning, container scanning y SAST.
- SBOM por release y firmas de artefactos.
- Prometheus metrics y OpenTelemetry traces donde aporte valor.
- Structured logs con redacción y correlation IDs.
- Dashboards operativos y alertas.
- CI backend/frontend/desktop/Android; iOS en runner macOS.
- Contract tests cross-language.
- Load tests de ingestión, task leasing y artifact upload.
- Chaos tests controlados.
- Backups, restore tests y disaster recovery objectives.
- No exponer Docker daemon sin TLS.

Tests obligatorios

- Security regression.
- Load targets.
- Agent storm.
- Redis/Postgres restart.
- Artifact store outage.
- Backup restore.
- CI clean checkout.

Documentación obligatoria

- Security baseline.
- Runbooks de alertas.
- SLOs/SLIs.
- Pipeline y release gate.
- Backup/DR.

Criterios de aceptación

- Builds reproducibles.
- Vulnerabilidades críticas bloquean release.
- Métricas permiten diagnosticar.
- Restore probado.

Forma de entrega

Antes de modificar código, inspeccioná `AGENTS.md`, ADRs, contratos, migraciones, ejemplos y tests relacionados. Presentá un plan breve. Implementá código completo, ejecutá las validaciones, corregí fallas y entregá un handoff con archivos, decisiones, comandos, resultados, riesgos y rollback.

Prompt 24 - Packaging, distribución y actualizaciones

Objetivo

Empaquetar servidor y agentes, firmar entregables y diseñar upgrades/rollback por plataforma.

Prerrequisitos

- Prompt 23 completo.

Alcance

- Server bundle.
- Windows installer.
- Linux packages.
- Android builds.
- iOS distribution.
- Update service.
- Rollback.

Requisitos de implementación

- Servidor: Compose versionado y opción Helm futura.
- Windows: MSI/MSIX o instalador aprobado, servicio, code signing y upgrade in-place.
- Linux: DEB y diseño RPM, systemd hardening.
- Android: APK/AAB firmado, canales interno/MDM/Play según estrategia.
- iOS: development/TestFlight/MDM; no prometer instalación desde Windows.
- Update manifest firmado con version, hash, compatibility y rollout ring.
- Desktop auto-update con descarga verificada y rollback.
- Mobile updates mediante canales de plataforma.
- DB migrations coordinadas con compatibility window.
- Canary/staged rollout y kill switch.
- Inventario de versiones y compliance.
- Uninstall y data retention policy.

Tests obligatorios

- Upgrade N-1 a N.
- Rollback.
- Interrupted install.
- Invalid signature.
- Old agent compatibility.
- Canary abort.
- Fresh install.

Documentación obligatoria

- Guías de instalación por plataforma.
- Release checklist.
- Update protocol.
- Rollback runbook.
- Matriz de soporte.

Criterios de aceptación

- Instalaciones reproducibles.
- Firmas verificadas.
- Rollback documentado.
- Servidor tolera versiones soportadas de agentes.

Forma de entrega

Antes de modificar código, inspeccioná `AGENTS.md`, ADRs, contratos, migraciones, ejemplos y tests relacionados. Presentá un plan breve. Implementá código completo, ejecutá las validaciones, corregí fallas y entregá un handoff con archivos, decisiones, comandos, resultados, riesgos y rollback.

Prompt 25 - Documentación, runbooks y soporte futuro

Objetivo

Completar documentación operativa y de desarrollo para que nuevas funciones puedan incorporarse sin depender del equipo original.

Prerrequisitos

- Todas las funciones principales disponibles.

Alcance

- Developer docs.
- Operator docs.
- API reference.
- Runbooks.
- Troubleshooting.
- Extension tutorials.
- Architecture index.

Requisitos de implementación

- Crear portal/docs navegable con MkDocs o equivalente.
- Documentar instalación, configuración, seguridad, backup, restore, upgrade y rollback.
- Referencia OpenAPI y schemas.
- Guías para agregar TestPlugin, TrafficGenerator, Adapter, CaptureProvider y ReportRenderer.
- Ejemplos completos y golden fixtures.
- Runbooks: agente offline, task stuck, server busy, high ingest lag, artifact failure, clock skew, mobile background, iperf server saturated.
- Troubleshooting por Windows/Linux/Android/iOS.
- Decision log y changelog.
- Matriz de compatibilidad y deprecaciones.
- Diagramas actualizados automáticamente o validados.
- Documentar límites conocidos y próximos pasos.
- Agregar templates para feature proposal, ADR, migration plan y test plan.

Tests obligatorios

- Link checker.
- Docs build.
- Code examples tested.
- Fresh-user walkthrough.
- Runbook tabletop exercise.

Documentación obligatoria

- Todo el alcance de esta fase es documentación.
- Agregar índice y ownership por sección.

Criterios de aceptación

- Un desarrollador nuevo puede añadir un plugin de ejemplo.
- Un operador puede diagnosticar fallas comunes.
- Los ejemplos compilan/pasan tests.
- No hay links rotos.

Forma de entrega

Antes de modificar código, inspeccioná `AGENTS.md`, ADRs, contratos, migraciones, ejemplos y tests relacionados. Presentá un plan breve. Implementá código completo, ejecutá las validaciones, corregí fallas y entregá un handoff con archivos, decisiones, comandos, resultados, riesgos y rollback.

Prompt 26 - Validación end-to-end y release candidate

Objetivo

Auditar la solución completa, ejecutar escenarios heterogéneos y producir una release candidate reproducible.

Prerrequisitos

- Prompts 01-25 completados.

Alcance

- E2E server.
- Four agent platforms.
- Traffic providers.
- Campaigns.
- Artifacts.
- Security.
- Backup/restore.
- Release.

Requisitos de implementación

- Levantar servidor en Windows 11 + WSL2 desde checkout limpio.
- Enrolar Windows, Linux, Android y iOS reales o la mejor combinación real/simulada documentada.
- Validar manifests distintos.
- Ejecutar latency, DNS, HTTP, iperf3 desktop, native mobile y latency-under-load.
- Ejecutar campaña heterogénea con PASS, FAIL, SKIPPED y BLOCKED.
- Forzar servidor offline, agente restart, app background y upload interrumpido.
- Verificar no duplicación.
- Generar reportes y comparar baseline.
- Validar revocación, RBAC y auditoría.
- Probar backup/restore y upgrade N-1.
- Ejecutar security scans y SBOM.
- Revisar documentación y known issues.
- Etiquetar release candidate solo si gates pasan.

Tests obligatorios

- E2E Playwright/API.
- Desktop integration.
- Android instrumented.
- iOS XCTest/device.
- Chaos.
- Load.
- Security.
- Restore.
- Upgrade.

Documentación obligatoria

- Release notes.
- Known issues.
- Evidence bundle.
- Compatibility matrix.
- Operations handoff.
- Go/no-go checklist.

Criterios de aceptación

- Instalación reproducible.
- Escenario E2E completo.
- No hay secretos ni vulnerabilidades críticas conocidas.
- Rollback y restore probados.
- Limitaciones están visibles.

Forma de entrega

Antes de modificar código, inspeccioná `AGENTS.md`, ADRs, contratos, migraciones, ejemplos y tests relacionados. Presentá un plan breve. Implementá código completo, ejecutá las validaciones, corregí fallas y entregá un handoff con archivos, decisiones, comandos, resultados, riesgos y rollback.